ACTIVITY GUIDE ON ANSIBLE

Contents

Practical 1: Install Ansible on Windows

Prerequisites

- A machine running Windows.
- A user account with administrator privileges.

Install Ansible on Windows Using WSL

Windows Subsystem for Linux (WSL) allows users to install different Linux distributions (such as Ubuntu) and use Linux apps and Bash command-line tools directly. This method does not require a virtual machine or dual boot setup.

Install Ansible with WSL by following the steps below:

1. Press the Windows key and type powershell. In the right panel, select the Run as administrator option.

2. Run the following command to install WSL and Ubuntu:

```
wsl --install
```

3. Once the installation completes, Ubuntu automatically launches. Run the following command to update the system repository information:

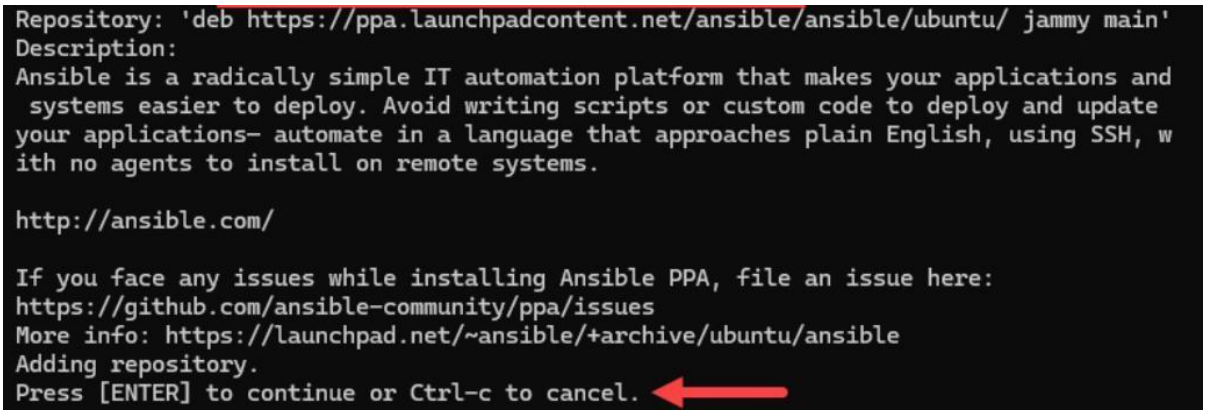
```
sudo apt update
```

4. Install the prerequisite packages that allow you to add the official Ansible PPA:

```
sudo apt install software-properties-common
```

5. Add the PPA with the following command:

```
sudo apt-add-repository ppa:ansible/ansible
```



6. Once again, update the package repository information after adding the PPA:

```
sudo apt update
```

7. Lastly, install Ansible by running:

```
sudo apt install ansible -y
```

Wait for the process to complete, and you can start using Ansible.

Practical 2: How to Generate and Set Up SSH Keys on Ubuntu

An SSH (Secure Shell) connection is essential for effectively managing a remote server. SSH keys, which consist of a public-private key pair, facilitate encrypted communication and serve as access credentials to establish a secure connection.

Prerequisites

- A user account with sudo privileges.
- Access to a terminal window/command line.

Generate SSH Key Pair

Generate a pair of SSH keys on the client system. The client system is the machine that connects to the SSH server.

1. Create a directory named .ssh in the home directory. The -p option ensures the system does not return an error if the directory exists:

```
mkdir -p $HOME/.ssh
```

2. Change permissions of the directory to give the user read, write, and execute privileges:

```
chmod 0700 $HOME/.ssh
cd .ssh
```

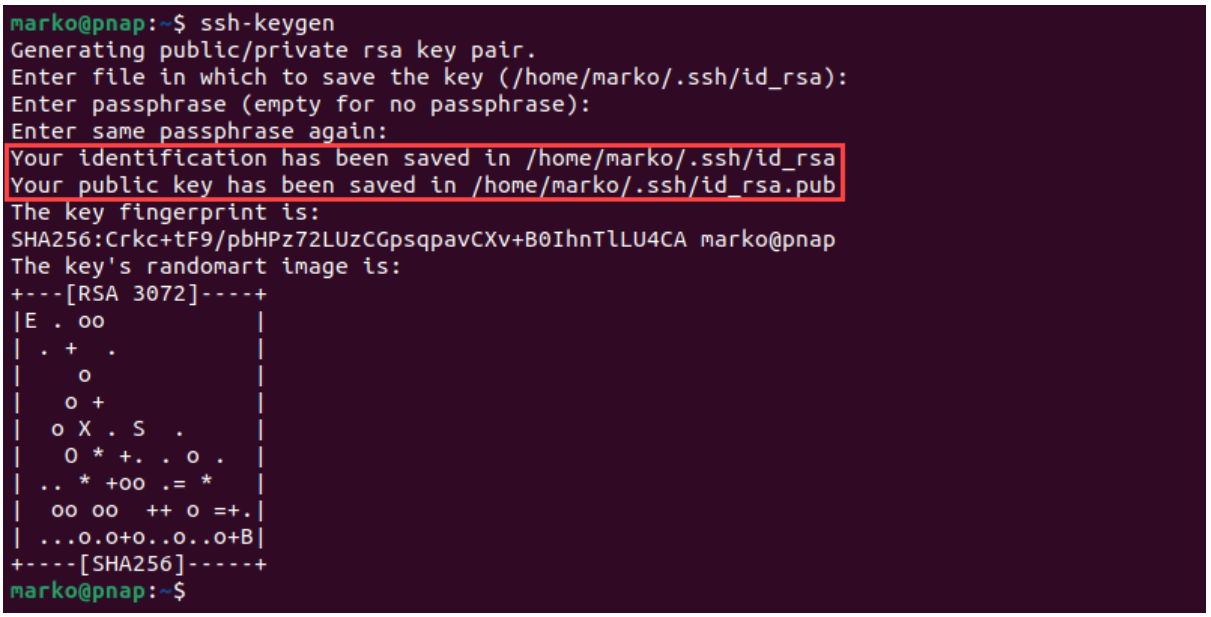
3. Execute the ssh-keygen command to create an RSA key pair:

```
ssh-keygen
```

4. When prompted, provide the path to the key file. If you press Enter without typing a file path, the key will be stored in the .ssh directory under the default file name id_rsa.

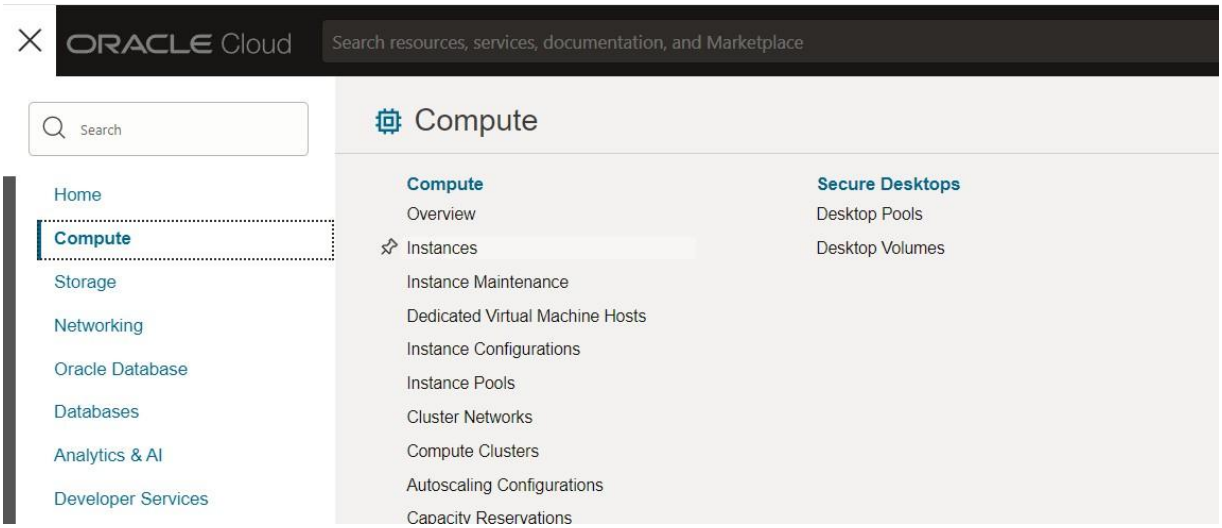
5. The system asks you to create a passphrase as an added layer of security. Input a memorable passphrase, and press Enter.

The output shows that the keys have been created successfully.

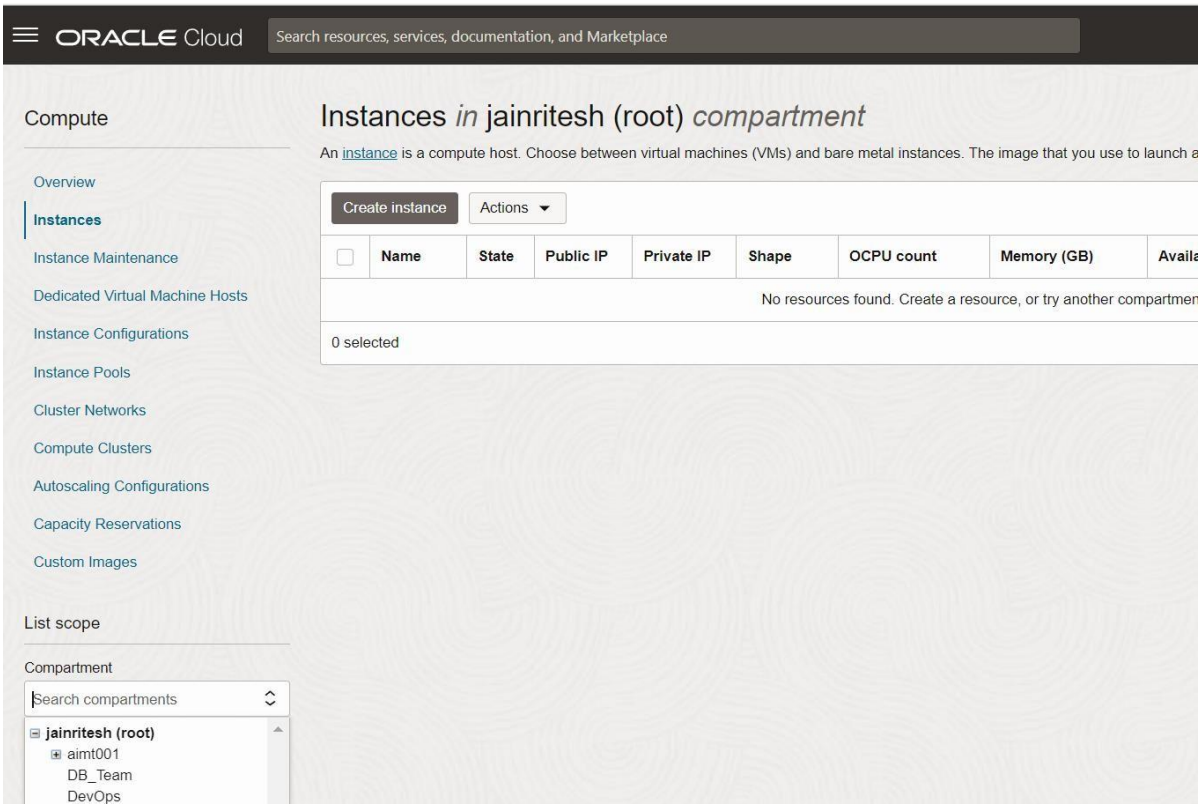

marko@pnap:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/marko/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/marko/.ssh/id_rsa
Your public key has been saved in /home/marko/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:Crkc+tF9/pbHPz72LUzCGpsqpavCXv+B0IhnTlLU4CA marko@pnap
The key's randomart image is:
+---[RSA 3072]----+
|E . oo |
| . + . |
| o |
| o + |
| o X . S . |
| O * +. . o . |
| .. * +oo .= * |
| oo oo ++ o =+.|
| ...o.o+o..o..o+B|
+----[SHA256]-----+
marko@pnap:~$

Practical 3: Creating Remote Instances in OCI

1. Navigate to Compute and click on Instances.



2. Select the compartment in which you want to create an instance.

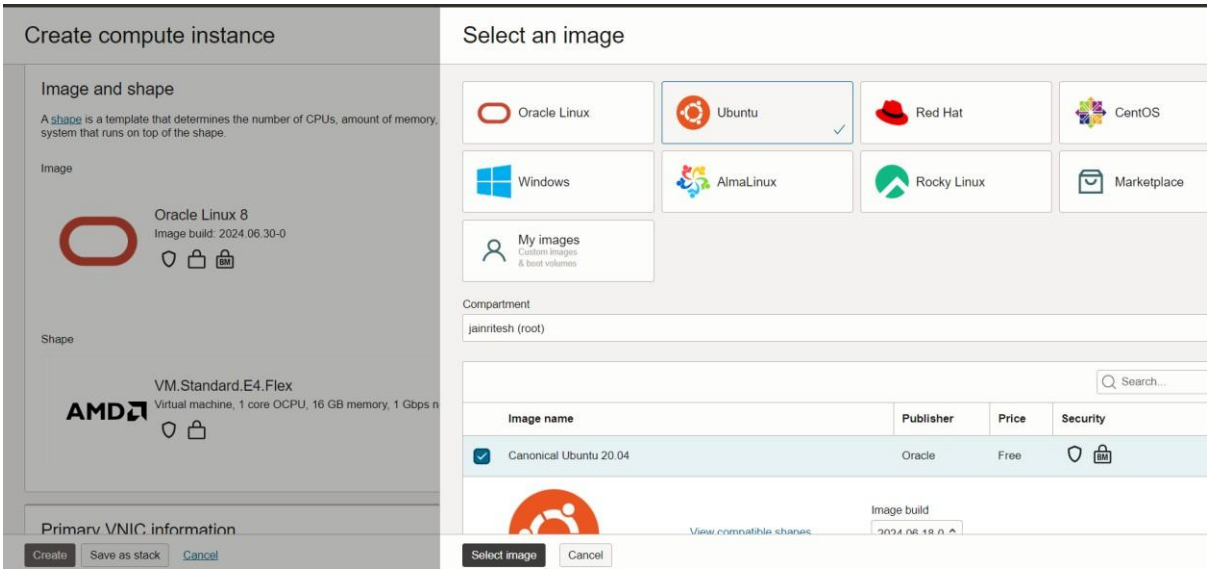


3. In the **Create Instance** dialog, Enter the following

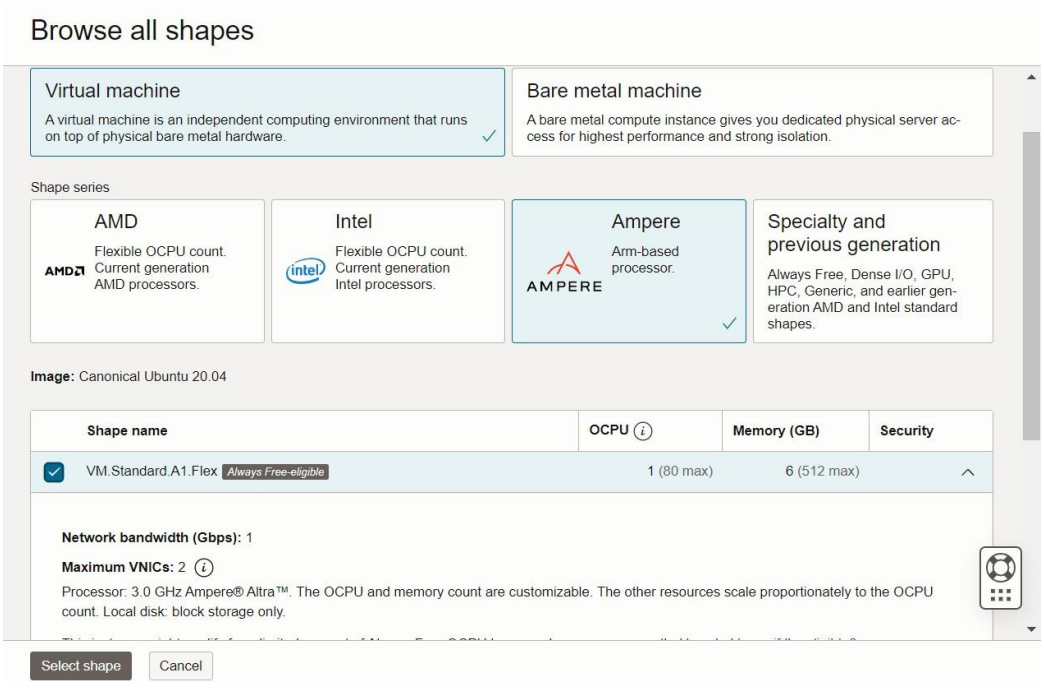
- **Name:** Manager-node-1

- **Compartment:** Select your assigned compartment
- **Availability-Domain**: AD-1

4. Select Ubuntu Image under Image and shape section



5. Select VM.standard.A1.Flex



6. Select the VCN and subnet details if you already have a VCN running if not select Create new virtual cloud network.

7. Select the Paste public key option in Add SSH-keys and give your public key which we generated in the previous lab.

8. Click on Create, and after some time your instance will be up and running.

Activity for Participants to create a Inventory and verify if the managed nodes are available or not using ping module.

Practical 4 – Executing Ansible Ad Hoc Commands

The objective of this practical is to learn how to execute **Ansible ad hoc commands** to perform administrative tasks on managed nodes without writing playbooks. You will use commonly used Ansible modules to gather information, manage files, install software, control services, and execute commands on remote systems.

Overview

Not every administrative task requires a playbook. Often, system administrators need to perform quick, one-time operations on one or more managed nodes.

Ansible provides **ad hoc commands**, which allow administrators to execute a single module against one or more managed nodes directly from the command line.

Ad hoc commands are commonly used for:

- Checking server availability
- Gathering system information
- Installing packages
- Managing services
- Creating directories
- Copying files
- Managing users
- Restarting services
- Running quick maintenance tasks

While playbooks are preferred for repeatable automation, ad hoc commands are ideal for immediate operational tasks.

Prerequisites

Before beginning this practical, ensure that:

- The managed node is reachable via SSH.
- The inventory file is configured correctly.
- The Ansible ping module returns pong.

What is an Ad Hoc Command?

An ad hoc command is a **single Ansible command** executed directly from the terminal to perform one task on one or more managed nodes.

General syntax:

ansible <host-pattern> -i <inventory> -m <module> -a "<arguments>"

Understanding the Syntax

Component	Description
ansible	Executes an ad hoc command
<host-pattern>	Specifies which managed hosts to target
-i	Specifies the inventory file
-m	Specifies the Ansible module
-a	Provides arguments to the module

Example:

ansible all -i inventory.ini -m ping

This command tells Ansible to:

1. Read the inventory.
2. Target all managed hosts.
3. Execute the ping module.
4. Display the results.

Step-by-Step Procedure

Step 1 – Verify Connectivity

Before executing any administrative tasks, confirm that the managed node is reachable.

```
ansible all -i inventory.ini -m ping
```

Expected output:

```
oraclelinux | SUCCESS => {
   "changed": false,
   "ping": "pong"
}
```

Explanation

This confirms that Ansible can authenticate and execute modules on the managed node.

Step 2 – Execute a Remote Command

Display the hostname of the managed node.

```
ansible all -i inventory.ini -m command -a "hostname"
```

Expected output:

```
oraclelinux | CHANGED | rc=0 >>
ansible-node-01
```

Explanation

The command module executes commands directly without invoking a shell. It is safer than the shell module because it does not interpret shell operators or environment variables.

Step 3 – Display Operating System Information

Run:

```
ansible all -i inventory.ini -m command -a "cat /etc/os-release"
```

Explanation

This retrieves operating system details from the managed node.

Step 4 – Display the Current Logged-in User

```
ansible all -i inventory.ini -m command -a "whoami"
```

Explanation

The command confirms the user Ansible uses to connect to the managed node. In this environment,

it should return opc.

Step 5 – Create a Directory
Create a directory named /tmp/ansible-demo.

ansible all -i inventory.ini -m file -a "path=/tmp/ansible-demo state=directory"

Explanation
The file module manages files, directories, permissions, and symbolic links. Setting state=directory ensures the directory exists.
Verify the directory:

ansible all -i inventory.ini -m command -a "ls -ld /tmp/ansible-demo"

Step 6 – Create a File
Create an empty file named /tmp/ansible-demo/demo.txt.

ansible all -i inventory.ini -m file -a "path=/tmp/ansible-demo/demo.txt state=touch"

Explanation
The touch state creates the file if it does not already exist.
Verify:
ansible all -i inventory.ini -m command -a "ls -l /tmp/ansible-demo"

Step 7 – Copy a File to the Managed Node
On the control node, create a sample file.

echo "Welcome to Ansible Practicals" > welcome.txt

Copy the file to the managed node.
ansible all -i inventory.ini -m copy -a "src=welcome.txt dest=/tmp/ansible-demo/welcome.txt"

Verify:
ansible all -i inventory.ini -m command -a "cat /tmp/ansible-demo/welcome.txt"

Explanation
The copy module transfers files from the control node to managed nodes while preserving file integrity.

Step 8 – Install a Package
Install the Apache HTTP Server.

ansible all -i inventory.ini -b -m dnf -a "name=httpd state=present"
Explanation

- -b enables privilege escalation (become), allowing Ansible to perform administrative tasks using sudo.
- The dnf module manages software packages on Oracle Linux.

- state=present ensures the package is installed.

Verify:

ansible all -i inventory.ini -m command -a "rpm -q httpd"

This command checks whether the Apache HTTP Server (httpd) package is installed on the managed node. It uses the Linux rpm command to query the RPM package database.

Step 9 – Start the Apache Service

ansible all -i inventory.ini -b -m service -a "name=httpd state=started enabled=yes"

Explanation

The service module manages system services.

- state=started starts the service if it is not already running.
- enabled=yes ensures the service starts automatically after a reboot.

Verify:

ansible all -i inventory.ini -m command -a "systemctl status httpd --no-pager"

This command displays the current status of the Apache HTTP Server (httpd) service on the managed node. It helps verify whether the service is running, stopped, or has encountered any errors.

Step 10 – Check the Apache Version

ansible all -i inventory.ini -m command -a "httpd -v"

Explanation

This confirms that Apache was installed successfully.

Step 11 – Display Disk Usage

ansible all -i inventory.ini -m command -a "df -h"

Explanation

This displays filesystem usage on the managed node.

Step 12 – Display Memory Information

ansible all -i inventory.ini -m command -a "free -m"

Practical 5 – Understanding Ansible Modules: command, shell, raw, and script

Objective

The objective of this practical is to understand the purpose and usage of the command, shell, raw, and script modules in Ansible. You will learn how each module executes commands on managed nodes, understand their differences, identify their limitations, and determine the appropriate module to use for different administrative tasks.

Overview

Ansible provides several modules for executing commands on managed nodes. Although these modules may appear similar, each serves a different purpose.

- The **command** module executes commands directly without invoking a shell.
- The **shell** module executes commands through the system shell and supports shell features such as pipes and redirection.
- The **raw** module executes commands directly over SSH without requiring Python on the managed node.
- The **script** module copies a local script from the control node to the managed node and executes it.

Selecting the appropriate module improves security, portability, and the overall reliability of automation.

Prerequisites

Before beginning this practical, ensure that:

- The managed node is reachable via SSH.
- The inventory file is configured correctly.
- Python is installed on the managed node (except when testing the raw module).
- The Ansible ping module returns **pong**.

Verify connectivity:

```
ansible all -i inventory.ini -m ping
```

Expected output:

```
web1 | SUCCESS => {
   "changed": false,
   "ping": "pong"
}
```

Step 1 – Execute a Simple Command Using the command Module

Display the hostname of the managed node.

```
ansible all -i inventory.ini -m command -a "hostname"
```

Expected output:

```
web1 | CHANGED | rc=0 >>
server01
```

Explanation

The command module executes the specified command directly on the managed node without invoking a shell. Since no shell is used, shell operators such as pipes (|), redirection (>), and environment variable expansion are not supported. This makes the command module more secure and is the preferred choice for executing simple commands.

Step 2 – Display Disk Usage

Run:

ansible all -i inventory.ini -m command -a "df -h"

Explanation

This command retrieves filesystem usage information from the managed node using the command module.

Step 3 – Observe the Limitation of the command Module

Execute the following command:

ansible all -i inventory.ini -m command -a "ps -ef | grep nginx"

Expected output:

FAILED | rc=1 >>

error: garbage option

Explanation

The command fails because the command module does not invoke a shell. Instead, the pipe (|) is passed directly as an argument to the ps command, which cannot interpret it. Shell operators such as pipes, redirection, and command chaining require the shell module.

Step 4 – Execute the Same Command Using the shell Module

Run:

ansible all -i inventory.ini -m shell -a "ps -ef | grep '[n]ginx'"

Explanation

The shell module executes commands through the system shell (/bin/sh). As a result, shell operators such as pipes, redirection, command substitution, and environment variables are processed correctly before execution.

Step 5 – Create a File Using Output Redirection

Execute:

ansible all -i inventory.ini -m shell -a "echo 'Welcome to Ansible' > /tmp/training.txt"

Verify:

ansible all -i inventory.ini -m command -a "cat /tmp/training.txt"

Explanation

The shell interprets the output redirection operator (>), creating the file and writing the specified text into it. This operation cannot be performed using the command module.

Step 6 – Execute a Command Using Environment Variables

Run:

ansible all -i inventory.ini -m shell -a "HOSTNAME=$(hostname); echo $HOSTNAME"

Explanation

The shell module supports environment variables and shell expansion, making it suitable for more complex command execution.

Step 7 – Execute a Command Using the raw Module

Run:

ansible all -i inventory.ini -m raw -a "hostname"

Expected output:

web1 | CHANGED | rc=0 >>

server01

Explanation

The raw module communicates directly over SSH and does not require Python on the managed node. It is primarily used during the initial provisioning of systems where Python has not yet been installed.

Step 8 – Install Python Using the raw Module
For Oracle Linux:
ansible all -i inventory.ini -m raw -a "sudo dnf install -y python3"
Verify:
ansible all -i inventory.ini -m ping
Explanation
Once Python is installed, all standard Ansible modules become available on the managed node.

Step 9 – Create a Local Script
On the Ansible control node, create a script named system_info.sh.

```
#!/bin/bash
echo "Hostname:"
hostname
echo
echo "Kernel Version:"
uname -r
echo
echo "Memory:"
free -m
echo
echo "Disk Usage:"
df -h
```

Make the script executable.
chmod +x system_info.sh

Explanation
The script gathers basic system information and will be executed remotely using the script module.

Step 10 – Execute the Script on the Managed Node
Run:
ansible all -i inventory.ini -m script -a "system_info.sh"
Explanation
The script module copies the script from the control node to the managed node, executes it, and returns the output. This module is useful when existing shell scripts need to be executed without manually copying them to each server.

Step 11 – Compare the Execution Modules

Feature	command	shell	raw	script
Uses a shell	No	Yes	Yes	Depends on script
Supports pipes (|)	No	Yes	Yes	Yes
Supports output redirection (>)	No	Yes	Yes	Yes
Supports environment variables	No	Yes	Yes	Yes
Requires Python	Yes	Yes	No	Yes
Copies local scripts	No	No	No	Yes
Recommended for	Simple commands	Complex shell commands	Initial server bootstrap	Executing reusable scripts

Practical 6: Creating the first Ansible Playbook

1. Create a project folder on your system.

```
mkdir ansible_quickstart && cd ansible_quickstart
```

Using a single directory structure makes it easier to add to source control as well as to reuse andshare automation content.

2. Create a file named inventory.ini in the ansible_quickstart directory that you created in thepreceding step.

Add a new [all] group to the inventory.ini file and specify the IP address or fully qualified domainname (FQDN) of each host system.

```
[all]
144.24.122.167 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
141.148.217.229 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Replace the *ip-address* field with the public ip address of your instances.

3. Create a file named playbook.yaml in your ansible_quickstart directory, that you created earlier,with the following content:

```
- name: My First play
  hosts: all
  tasks:
    - name: Ping my hosts
      ansible.builtin.ping:

    - name: Print Message
      ansible.builtin.debug:
        msg: Hello World
```

4. To test the connectivity whether Ansible can connect to the hosts using the following command:

```
adminu@DESKTOP-I8CGSGS:~/ansible_quick$ ansible -i inventory.ini all -m ping
141.148.217.229 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
144.24.122.167 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

5. Run your playbook.

```
ansible-playbook -i inventory.ini playbook.yaml
```

Ansible returns the following output:

```
adminu@DESKTOP-I8CGSGS:~/ansible_quick$ ansible-playbook -i inventory.ini playbook.yaml

PLAY [My First play] ***********************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************
ok: [141.148.217.229]
ok: [144.24.122.167]

TASK [Ping my hosts] ***********************************************************************************************************************
ok: [141.148.217.229]
ok: [144.24.122.167]

TASK [Print Message] ***********************************************************************************************************************
ok: [144.24.122.167] => {
    "msg": "Hello World"
}
ok: [141.148.217.229] => {
    "msg": "Hello World"
}

PLAY RECAP *********************************************************************************************************************************
141.148.217.229            : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
144.24.122.167             : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

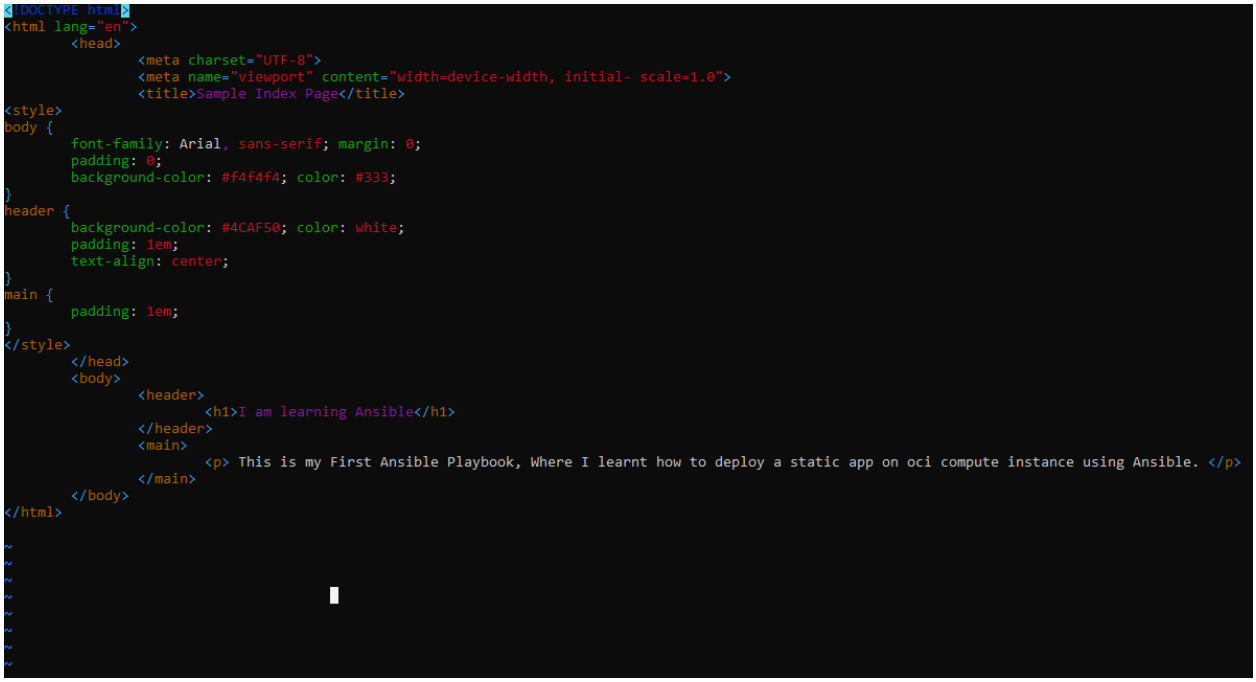
Practice 7: Deploying a static App on a compute instance

1. Make sure that the instance is running and the inventory.ini file is present within the directory that you created earlier.
Create the index.html step as mentioned in step 3 before creating the playbook

2. Create another playbook and give a **"playbook1.yaml "name**

```
---
- hosts: all
  become: true
  tasks:
    - name: Install apache httpd
      ansible.builtin.dnf:
        name: apache2
        state: present
        update_cache: yes
    - name: Copy file with owner and permissions
      ansible.builtin.copy:
        src: index.html
        dest: /var/www/html
        owner: root
        group: root
        mode: '0644'
```

3. Create a static html page within the same directory. **(vi index.html)**

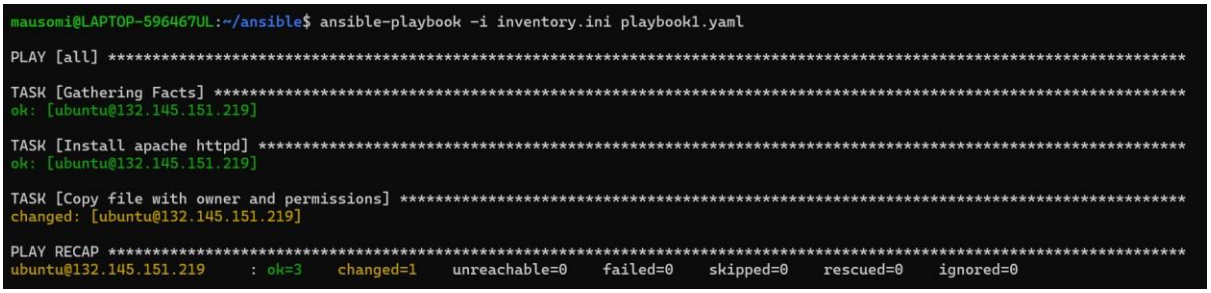


```
<!DOCTYPE html>
<html lang="en">
        <head>
                <meta charset="UTF-8">
                <meta name="viewport" content="width=device-width, initial- scale=1.0">
                <title>Sample Index Page</title>
<style>
body {
        font-family: Arial, sans-serif; margin: 0;
        padding: 0;
        background-color: #f4f4f4; color: #333;
}
header {
        background-color: #4CAF50; color: white;
        padding: 1em;
        text-align: center;
}
main {
        padding: 1em;
}
</style>
        </head>
        <body>
                <header>
                        <h1>I am learning Ansible</h1>
                </header>
                <main>
                        <p> This is my First Ansible Playbook, Where I learnt how to deploy a static app on oci compute instance using Ansible. </p>
                </main>
        </body>
</html>
```

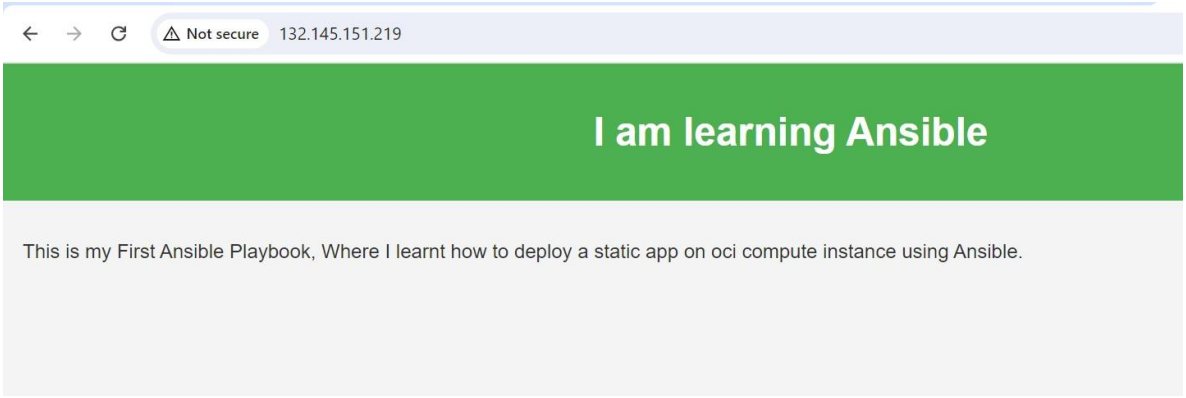
4. Run the playbook

```
ansible-playbook -i inventory.ini playbook1-name.yaml
```

The output will look like some thing this



Paste the public ip address in the browser



Configure Network Access for the Web Server (Important)

After successfully installing and starting the Apache HTTP Server, ensure that the web server is accessible from outside the compute instance.

1. Allow HTTP Traffic in OCI Security List

By default, an OCI Security List allows SSH (port 22) but blocks HTTP (port 80). To access the web application from a browser, create an ingress rule that allows TCP traffic on port **80**.

Navigate to:

OCI Console → Networking → Virtual Cloud Networks → Your VCN → Subnet → Security List → Add Ingress Rule

Configure the rule as follows:

Parameter	Value
Source CIDR	0.0.0.0/0
IP Protocol	TCP
Source Port Range	All
Destination Port Range	80
Stateless	No

Click **Add Ingress Rule** to save the configuration.

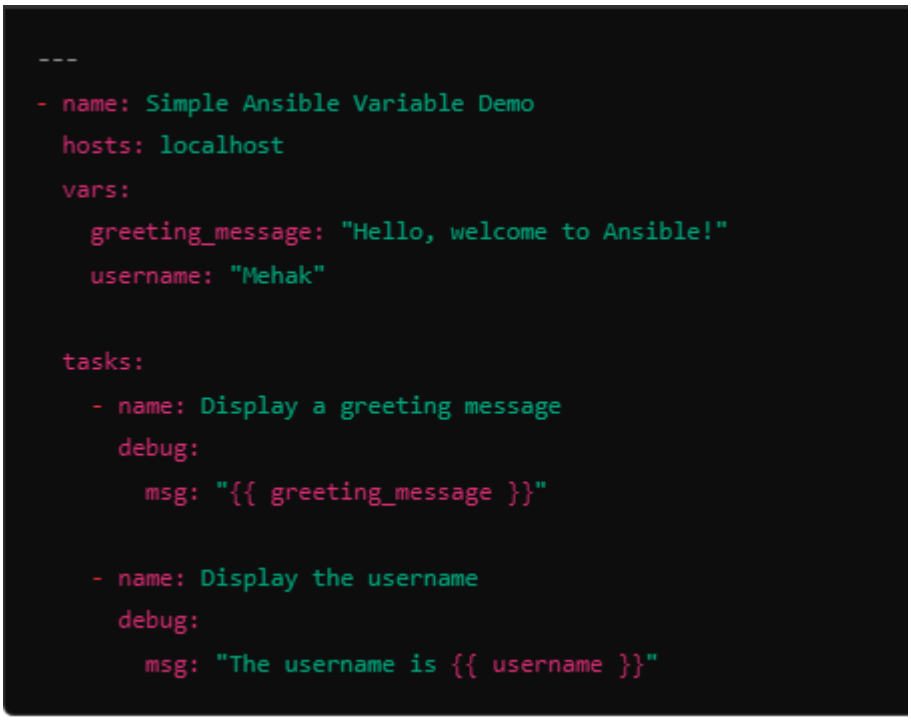
Practice 8: Exploring Variables

Tasks:

step 1 : Open a text editor and create a new file called simple_var_demo.yml. You can use any editor like Notepad, VSCode, or Nano (in Linux)

step 2 :

```
---
- name: Simple Ansible Variable Demo
  hosts: localhost
  vars:
    greeting_message: "Hello, welcome to Ansible!"
    username: "Mehak"

  tasks:
    - name: Display a greeting message
      debug:
        msg: "{{ greeting_message }}"

    - name: Display the username
      debug:
        msg: "The username is {{ username }}"
```



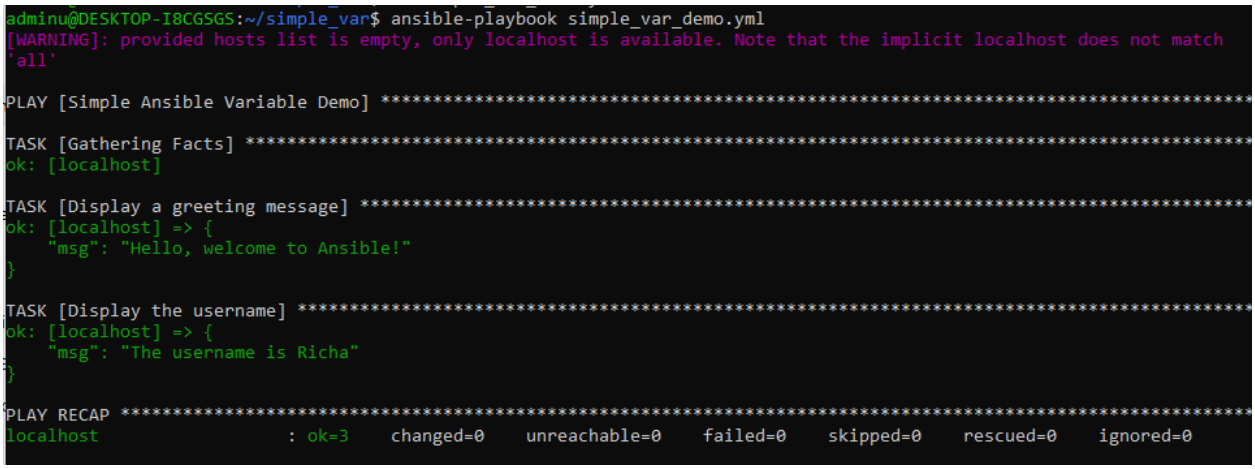
Explanation of the playbook:

- **hosts: localhost** – Tells Ansible to run the playbook on your local machine.
- **vars:** – Defines variables that you can reuse in tasks (greeting_message and username).
- **tasks:** – Executes actions (here, we use debug to print messages to the screen).

Step 3: Run the Playbook

- Open your terminal in the folder where the simple_var_demo.yml file is saved.
- Run this command:

```
ansible-playbook simple_var_demo.yml
```



Practice 9: Creating a Simple role and using it in a playbook

1. Ensure the instance is running and the inventory.ini file is up-to-date.

2. Navigate to the folder which we created in the previous lab.

3. Ensure that you have the **index.html**, **inventory.ini** and **playbook.yaml** file is present.

4. To create the roles structure execute the following:

```
ansible-galaxy init role-name
```

```
mausomi@LAPTOP-596467UL:~/ansible$ ansible-galaxy init httpd-role
- Role httpd-role was created successfully
```

```
mausomi@LAPTOP-596467UL:~/ansible$ ls -ltr httpd-role
total 36
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 templates
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 files
-rw-r--r-- 1 mausomi mausomi 1328 Aug 11 10:17 README.md
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 vars
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 tasks
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 defaults
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 tests
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 meta
drwxr-xr-x 2 mausomi mausomi 4096 Aug 11 10:17 handlers
```

You can see that the whole folder structure has been created.

5. If you check your playbook we only have tasks and one file so we have put them into the correct folder.

6. Copy the tasks section and put it into ***tasks/main.yml*** file

```
 GNU nano 6.2                                    httpd-role/tasks/main.yml
---
# tasks file for httpd-role
   - name: Install apache httpd
     ansible.builtin.apt:
       name: apache2
       state: present
       update_cache: yes
   - name: Copy file with owner and permissions
     ansible.builtin.copy:
       src: files/index.html
       dest: /var/www/html
       owner: root
       group: root
       mode: '0644'
```

Your main.yml file should look like this.

7. Update the playbook file accordingly.

```
---
- hosts: all
  become: true
  roles:
   - httpd-role
```

You have to specify your role name in the roles field.

8. Similarly, you have to move your index.html file to *httpd-role/files/*

```
mausomi@LAPTOP-596467UL:~/ansible$ mv index.html httpd-role/files/
mausomi@LAPTOP-596467UL:~/ansible$ ls
httpd-role  inventory.ini  playbook1.yaml  playbook2.yaml
mausomi@LAPTOP-596467UL:~/ansible$ cd httpd-role
mausomi@LAPTOP-596467UL:~/ansible/httpd-role$ ls
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
mausomi@LAPTOP-596467UL:~/ansible/httpd-role$ cd files
mausomi@LAPTOP-596467UL:~/ansible/httpd-role/files$ ls
index.html
```

As we have moved the ***index.html*** file we have to put the correct location in the ***tasks/main.yaml*** file.

9. Specify the location in the main.yml and it should look somewhat like this:

```
---
# tasks file for httpd-role
    - name: Install apache httpd
      ansible.builtin.apt:
        name: apache2
        state: present
        update_cache: yes
    - name: Copy file with owner and permissions
      ansible.builtin.copy:
        src: files/index.html
        dest: /var/www/html
        owner: root
        group: root
        mode: '0644'
```

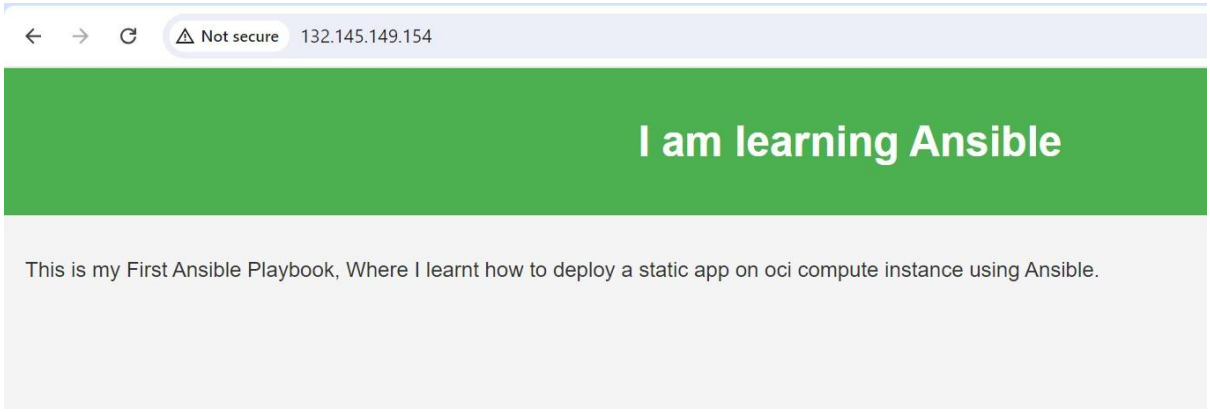
You can see that the ***src*** location has been updated.

10. Execute your playbook.

```
ansible-playbook -i inventory.ini playbook-name.yaml
```

```
mausomi@LAPTOP-596467UL:~/ansible$ ansible-playbook -i inventory.ini playbook1.yaml

PLAY [all] *********************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************
ok: [ubuntu@132.145.149.154]

TASK [httpd-role : Install apache httpd] ***************************************************************************************
changed: [ubuntu@132.145.149.154]

TASK [httpd-role : Copy file with owner and permissions] ***********************************************************************
changed: [ubuntu@132.145.149.154]

PLAY RECAP *********************************************************************************************************************
ubuntu@132.145.149.154     : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

11. Copy and paste the public ip address of the instance and your application has worked properly.



Practice 10: In this practice, you will learn how to use Ansible Galaxy to install and use pre-built roles from the community.

Specifically, we will install a role for setting up the Apache web server on a target machine and then apply this role in a playbook to automate the installation process

Pre-Requisites

- Ansible should be installed on your system
- The targeted nodes should be configured

Tasks

1. create a directory and move to that directory
   mkdir Ansible-galaxy-demo
   cd Ansible-galaxy-demo

2. Install a role from Ansible Galaxy

ansible-galaxy install geerlingguy.apache

3. create a playbook (vi install_apache.yaml)

```
---
- name: install Apache Web Server
  hosts: myservers  # Change this to your target hosts
  become: yes  # Use sudo to install packages
  roles:
    - geerlingguy.apache  # Use the installed role

~
```

4. create an inventory file (vi inventory.ini)

```
[myservers]
server1 ansible_host=141.148.217.229 ansible_user=ubuntu
```

5. Run a playbook

ansible-playbook -i inventory.ini install_apache.yml

```
TASK [geerlingguy.apache : Update apt cache.] *********************************************************************
changed: [server1]

TASK [geerlingguy.apache : Ensure Apache is installed on Debian.] *************************************************
ok: [server1]

TASK [geerlingguy.apache : Get installed version of Apache.] ******************************************************
ok: [server1]

TASK [geerlingguy.apache : Create apache_version variable.] *******************************************************ok: [server1]

TASK [geerlingguy.apache : Include Apache 2.2 variables.] *********************************************************skipping: [server1]

TASK [geerlingguy.apache : Include Apache 2.4 variables.] *********************************************************ok: [server1]

TASK [geerlingguy.apache : Configure Apache.] **********************************************************************included: /home/adminu/.ansible/roles/geerlingguy.apache/tasks/configure-Debian.yml for server1

TASK [geerlingguy.apache : Configure Apache.] **********************************************************************ok: [server1] => (item={'regexp': '^Listen ', 'line': 'Listen 80'})

TASK [geerlingguy.apache : Enable Apache mods.] ********************************************************************changed: [server1] => (item=rewrite)
changed: [server1] => (item=ssl)

TASK [geerlingguy.apache : Disable Apache mods.] *******************************************************************skipping: [server1]

TASK [geerlingguy.apache : Check whether certificates defined in vhosts exist.] ************************************skipping: [server1]

TASK [geerlingguy.apache : Add apache vhosts configuration.] *******************************************************changed: [server1]

TASK [geerlingguy.apache : Add vhost symlink in sites-enabled.] ****************************************************changed: [server1]

TASK [geerlingguy.apache : Remove default vhost in sites-enabled.] *************************************************skipping: [server1]

TASK [geerlingguy.apache : Ensure Apache has selected state and enabled on boot.] **********************************ok: [server1]

RUNNING HANDLER [geerlingguy.apache : restart apache] **************************************************************changed: [server1]

PLAY RECAP *********************************************************************************************************server1                    : ok=16   changed=5
  unreachable=0    failed=0    skipped=5    rescued=0    ignored=0
```

Practice 11: To demonstrate the usage of Ansible collections by creating a simple playbook that pings remote servers to verify connectivity.

Prerequisites:

- Ansible installed on your local machine.
- Access to remote servers listed in the inventory file.
- A configured inventory file (`inventory.ini`) with the IP addresses of the target servers.
- Ansible collections installed (if required for specific tasks).

Tasks

1. **Create a directory and move to that directory**

   mkdir Ansible-collection-demo

   cd Ansible-collection-demo

2. **Create an inventory file inventory.ini**

```
[myservers]

Server1 ansible_host=141.148.217.229 ansible_user=ubuntu
```

3. **Create a playbook named demo-collection.yaml**

```
---
- name: Demo Playbook to Use Ansible Collection
  hosts: myservers
  tasks:
   - name: Ping the server using ansible.builtin.ping
     ansible.builtin.ping:
~
~
~
~
~
```

4. **Run the playbook**

   ansible-playbook -i inventory.ini demo-collection.yaml

```
adminu@DESKTOP-I8CGSGS:~/ansible-collection-demo$ ansible-playbook -i inventory.ini demo-collection.yaml

PLAY [Demo Playbook to Use Ansible Collection] *********************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [Server1]

TASK [Ping the server using ansible.builtin.ping] ******************************************************
ok: [Server1]

PLAY RECAP *********************************************************************************************
Server1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Here is a breakdown of the output:

- **Gathering Facts:** The playbook first gathers system information (facts) about the target server(s). This is typical behavior in Ansible, unless explicitly disabled.
- **Ping Task:** The task "Ping the server using ansible.builtin.ping" was executed successfully, and you can see the output ok: [Server1], indicating that the ping succeeded.
- PLAY RECAP:
    - **ok=2:** Two tasks were executed successfully (gathering facts and the ping task).
    - **changed=0**: No changes were made to the server, which is expected since ping does not modify anything.
    - **unreachable=0**, failed=0: No servers were unreachable, and no tasks failed.
    - **skipped=0**, rescued=0, ignored=0: No tasks were skipped, and no error-handling steps were needed.

Practice 12 – Working with Ansible Variables and Facts

Objective

The objective of this practical is to learn how to use variables and facts in Ansible playbooks. You will define variables, use them to create reusable playbooks, gather system information automatically using Ansible facts, register task outputs, and use conditional statements to control task execution.

Overview

Hardcoding values such as package names, file paths, ports, or usernames makes playbooks difficult to reuse and maintain. Ansible variables allow these values to be defined once and referenced throughout a playbook, making automation more flexible.

In addition to user-defined variables, Ansible automatically collects **facts**, which are system properties such as the hostname, operating system, IP address, CPU information, and memory details. These facts enable playbooks to make intelligent decisions based on the characteristics of the managed node.

Variables and facts are fundamental building blocks for creating dynamic and reusable automation.

Prerequisites

Before beginning this practical, ensure that:

- The managed node is reachable via SSH.
- The inventory file is configured correctly.
- The Ansible ping module returns **pong**.
- Python is installed on the managed node.

Verify connectivity:

```
ansible all -i inventory.ini -m ping
```

Expected output:

```
web1 | SUCCESS => {
   "changed": false,
   "ping": "pong"
}
```

Understanding Variables

A variable is a named value that can be referenced throughout a playbook.

Instead of hardcoding values:

```
name: httpd
```

you can define a variable:

```
package_name: httpd
```

and reference it using:

name: "{{ package_name }}"

Using variables makes playbooks reusable across multiple environments.

Step-by-Step Procedure
Step 1 – Create a Playbook with Variables
Create a playbook named variables.yml.

vi variables.yml

Add the following content:

```
---

- name: Variable Demonstration


  hosts: all


  vars:

    package_name: httpd


  tasks:


    - name: Install Apache

      dnf:

        name: "{{ package_name }}"

        state: present

      become: yes
```

Execute:

ansible-playbook -i inventory.ini variables.yml

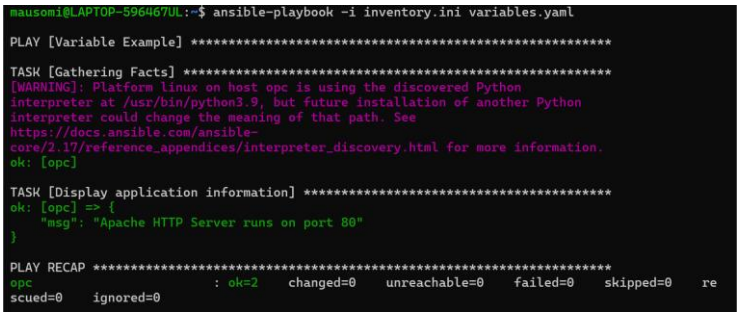
Explanation
The variable package_name stores the package name. Instead of directly specifying httpd, the playbook references the variable using Jinja2 syntax ({{ }}).

Step 2 – Define Multiple Variables
Modify the playbook.

```
---

- name: Variable Example


  hosts: all


  vars:

    app_name: Apache HTTP Server

    package_name: httpd

    service_name: httpd

    service_port: 80


  tasks:


    - name: Display application information

      debug:

        msg: "{{ app_name }} runs on port {{ service_port }}"
```

Execute:

```
ansible-playbook -i inventory.ini variables.yml
```



Explanation
The debug module displays variable values during playbook execution.

Step 3 – Pass Variables at Runtime
Create a playbook named runtime-variable.yml.

```
---

- name: Runtime Variable Example


  hosts: all


  tasks:


    - name: Display package name

      debug:

        msg: "Installing {{ package_name }}"
```

Execute the playbook while supplying the variable.

ansible-playbook -i inventory.ini runtime-variable.yml -e "package_name=git"

Expected output:

Installing git

Explanation
The -e option passes variables during playbook execution without modifying the playbook.

Understanding Ansible Facts
Facts are system properties automatically collected by Ansible before task execution.
Examples include:

- Hostname
- Operating System
- Kernel Version
- CPU Architecture
- Memory
- IP Address
- Network Interfaces

Facts allow playbooks to adapt automatically to different systems.

Step 4 – View All Available Facts
Run:

```
ansible all -i inventory.ini -m setup
```

Explanation
The setup module gathers all facts from the managed node and displays them in JSON format.

Step 5 – Display the Hostname Using Facts
Create a playbook named facts.yml.

```
---

- name: Display Hostname


  hosts: all


  tasks:


    - name: Display hostname

      debug:

        msg: "Hostname: {{ ansible_hostname }}"
```

Execute:

```
ansible-playbook -i inventory.ini facts.yml
```

Expected output:

Hostname: server01

Explanation
ansible_hostname is an automatically collected fact representing the hostname of the managed node.

Step 6 – Display Operating System Information
Modify the playbook.

```
---

- name: Display Operating System


  hosts: all


  tasks:


    - name: Show OS information

      debug:

        msg: "{{ ansible_distribution }} {{ ansible_distribution_version }}"
```

Execute:

ansible-playbook -i inventory.ini facts.yml

Expected output:

OracleLinux 9.5

Explanation
The facts ansible_distribution and ansible_distribution_version identify the operating system.

Step 7 – Display Network Information
Modify the playbook.

```
---

- name: Display Network Information
```

hosts: all

tasks:

- name: Show IP address

debug:

msg: "{{ ansible_default_ipv4.address }}"

Execute:

ansible-playbook -i inventory.ini facts.yml

Explanation
The fact ansible_default_ipv4.address displays the primary IP address of the managed node.

Step 8 – Register Command Output
Create a playbook named register.yml.

```
---
- name: Register Output

  hosts: all

  tasks:

    - name: Check uptime
      command: uptime
      register: uptime_result

    - name: Display uptime
      debug:
        var: uptime_result.stdout
```

Execute:

ansible-playbook -i inventory.ini register.yml

Expected output:

```
TASK [Gathering Facts] *********************************************************
[WARNING]: Platform linux on host opc is using the discovered Python
interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [opc]

TASK [Check uptime] ************************************************************
changed: [opc]

TASK [Display uptime] **********************************************************
ok: [opc] => {
    "uptime_result.stdout": " 18:26:21 up  3:24,  1 user,  load average: 0.07, 0.02, 0.00"
}

PLAY RECAP *********************************************************************
opc                        : ok=3    changed=1    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0
```

Output	Explanation
TASK [Display uptime]	Indicates that Ansible is executing the second task, which displays the registered variable.
ok: [opc]	The task executed successfully on the managed node opc.
uptime_result.stdout	uptime_result is the variable created using the register keyword, and stdout contains the standard output of the uptime command.
18:26:21	Current system time on the managed node.
up 3:24	The server has been running continuously for 3 hours and 24 minutes since its last boot.
1 user	One user is currently logged into the system.
load average: 0.07, 0.02, 0.00	The average CPU load over the last 1 minute, 5 minutes, and 15 minutes, respectively. These values are very low, indicating the system is under minimal load.

Explanation

The register keyword is used when you want to **capture the output of a task** so that it can be referenced

by subsequent tasks in the playbook.

Step 9 – Display the Return Code
Modify the second task.

- name: Display return code

  debug:

    var: uptime_result.rc

```yaml
---
- name: Register Output
  hosts: all

  tasks:
    - name: Check uptime
      ansible.builtin.command: uptime
      register: uptime_result

    - name: Display return code
      ansible.builtin.debug:
        var: uptime_result.rc
```

Explanation
Registered variables contain multiple attributes, including:

Attribute	Description
stdout	Standard output
stderr	Error output
rc	Return code
changed	Indicates whether the task changed the system

Step 10 – Execute a Task Conditionally
Create a playbook named conditional.yml.

```
---
- name: Conditional Execution

  hosts: all

  tasks:

    - name: Install Apache on Oracle Linux
      dnf:
        name: httpd
        state: present
      become: yes
      when: ansible_distribution == "OracleLinux"
```

Execute

```
ansible-playbook -i inventory.ini conditional.yml
```

The when statement evaluates the condition before executing the task. If the condition evaluates to true, the task is executed; otherwise, it is skipped.

Step 11 – Disable Fact Gathering

YAML

```yaml
---
- name: Disable Fact Gathering

  hosts: all

  gather_facts: false

  tasks:

    - name: Display hostname
      command: hostname
```

Execute

```
ansible-playbook -i inventory.ini nofacts.yml
```

By default, Ansible gathers facts before executing tasks. Setting gather_facts: false improves execution time when system facts are not required.

Best Practices

- Use variables instead of hardcoded values.
- Choose meaningful variable names.
- Use runtime variables for environment-specific values.
- Gather facts only when necessary.
- Register outputs that will be reused later.
- Use conditional statements to make playbooks adaptable across multiple operating systems.

Practical 13 – Working with Ansible Handlers

Objective

The objective of this practical is to learn how Ansible handlers work and how they help automate service management efficiently. You will understand the concept of event-driven automation, use the notify directive to trigger handlers, and restart services only when configuration changes occur.

Overview

In system administration, configuration changes often require a service to be restarted before the changes become effective. For example:

- After modifying an Apache configuration file, the Apache service must be restarted.
- After changing an SSH configuration, the SSH service must be restarted.
- After updating an NGINX configuration, the NGINX service must be restarted.

A common mistake is restarting services after every playbook execution, even when no changes have been made. This increases downtime and consumes unnecessary system resources.

Ansible solves this problem using **Handlers**.

A handler is a special type of task that runs **only when another task notifies it**. If no changes occur, the handler is skipped automatically.

This makes Ansible automation efficient, idempotent, and suitable for production environments.

Prerequisites

Before beginning this practical, ensure that:

- The managed node is reachable.
- Apache HTTP Server (httpd) is installed.
- The Apache service is running.
- The inventory file is configured correctly.
- The Ansible ping module returns **pong**.

Verify connectivity.

```
ansible all -i inventory.ini -m ping
```

Why Do We Need Handlers?

Consider the following playbook.

```
- name: Copy configuration

  copy:

    src: httpd.conf

    dest: /etc/httpd/conf/httpd.conf



- name: Restart Apache

  service:

    name: httpd
```

state: restarted

Suppose the configuration file has **not changed**.
Even then, the Apache service is restarted every time the playbook runs.
This is inefficient because:

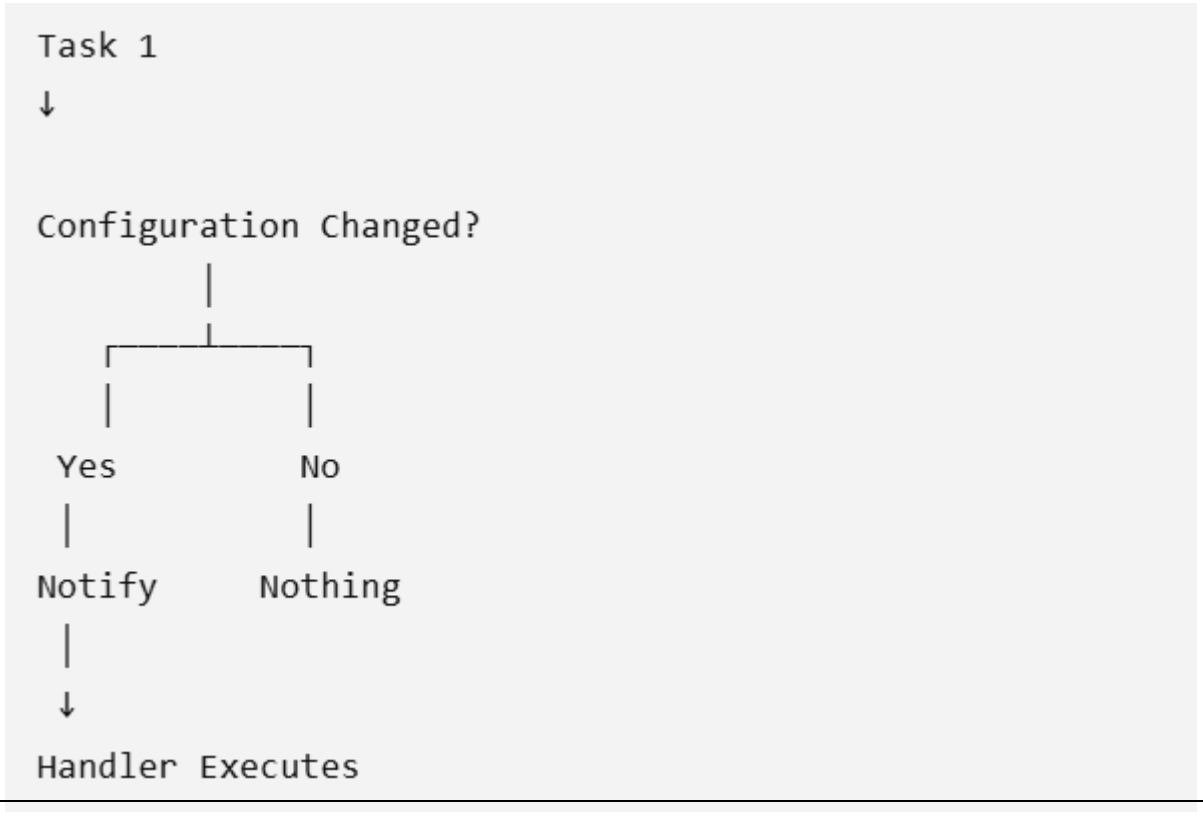
- Existing client connections may be interrupted.
- Restarting services consumes system resources.
- Large production environments may experience unnecessary downtime.

Instead, the service should restart **only when the configuration file changes**.
Handlers provide this capability.

Understanding Handler Execution
A normal task executes every time.

```
Playbook Execution

Task 1
↓

Task 2
↓

Task 3
↓

Restart Service
```

Handler Executes



This is known as **event-driven execution**.

Step-by-Step Procedure
Step 1 – Create a New Playbook
Create a file named:

vi handler-demo.yml

Explanation
This playbook demonstrates how handlers respond to configuration changes.
Unlike previous playbooks, this playbook contains two additional sections:

- notify
- handlers

These two sections work together to provide event-driven automation.

Step 2 – Create a Simple Configuration File
On the Ansible control node, create a file named index.html.

vi index.html

Add the following content.

<h1>Welcome to Ansible Training</h1>

Save the file.
Explanation
This HTML file will be copied to the Apache web server.
Initially, it contains only a simple heading.
Later in this practical, you will modify this file to observe how handlers react to configuration changes.

Step 3 – Create the Playbook
Add the following content to handler-demo.yml.

```
---

- name: Deploy Web Page


  hosts: all


  become: yes
```

```
 tasks:


   - name: Copy index page

     copy:

       src: index.html

       dest: /var/www/html/index.html

     notify: Restart Apache


 handlers:


   - name: Restart Apache

     service:

       name: httpd

       state: restarted
```

Explanation

Let's understand each section of this playbook.

The play targets all managed hosts.

```
hosts: all
```

Since copying files into /var/www/html requires administrative privileges, privilege escalation is enabled.

```
become: yes
```

The copy module copies the local file from the Ansible control node to the managed node.

```
copy:

  src: index.html

  dest: /var/www/html/index.html
```

The important line is:

```
notify: Restart Apache
```

This does **not** restart Apache immediately.

Instead, it tells Ansible:
"If this task changes the managed node, remember to execute the handler named **Restart Apache** after all normal tasks have completed."

Step 4 – Execute the Playbook
Run:

ansible-playbook -i inventory.ini handler-demo.yml

Expected output:

TASK [Copy index page]

changed: [web1]

RUNNING HANDLER [Restart Apache]

changed: [web1]

Explanation
Because the file was copied for the first time, the managed node changed.
As a result:

- The copy task reports changed.
- The notification is sent.
- The handler executes.
- Apache is restarted.

Notice that handlers execute **after all regular tasks**, not immediately after the notifying task.

If the file is already there , delete the file using the below command and then run the playbook again.

ansible all -i inventory.ini -b -m file -a "path=/var/www/html/index.html state=absent"

Step 5 – Execute the Playbook Again
Run the same playbook again.

ansible-playbook -i inventory.ini handler-demo.yml

Expected output:

TASK [Copy index page]

ok: [web1]

PLAY RECAP

web1 : ok=2 changed=0 failed=0

Explanation
This time, the source and destination files are identical.
Since no changes occurred:

- The copy module reports ok.
- No notification is sent.
- The handler does not execute.
- Apache is **not restarted**.
- This demonstrates **idempotency**.

Step 6 – Modify the Configuration File
Open index.html.

vi index.html

Change the content to:

<h1>Welcome to OCI DevOps Training</h1>

Save the file.
Explanation
The local file now differs from the version deployed on the managed node.
During the next execution, the copy module detects this difference and replaces the remote file.
Since the task modifies the managed node, it will notify the handler.

Step 7 – Execute the Playbook Again
Run:

ansible-playbook -i inventory.ini handler-demo.yml

Expected output:

TASK [Copy index page]

changed: [web1]

RUNNING HANDLER [Restart Apache]

changed: [web1]

Explanation
Because the content changed, the copy task reports changed.
The handler is notified and Apache is restarted automatically.
This behavior ensures services restart **only when necessary**.

Step 8 – Verify the Updated Web Page
Access the Apache web server from a browser.

http://<Managed_Node_IP>

You should see:

Welcome to OCI DevOps Training

Explanation
The updated file has been copied successfully, and the handler ensured Apache reloaded the new content.

Best Practices

- Use handlers for restarting or reloading services.
- Avoid restarting services as regular tasks.
- Give handlers descriptive names.
- Reuse handlers across multiple tasks by using the same notify name.
- Use handlers to minimize unnecessary downtime in production environments.

Practical 14 – Managing Configuration Files Using Ansible Templates (Jinja2)

Objective

The objective of this practical is to learn how to create and deploy dynamic configuration files using Ansible Templates. You will create Jinja2 template files, use variables and Ansible facts within templates, deploy templates to managed nodes using the template module, and integrate templates with handlers to automatically restart services whenever configuration changes occur.

Overview

In a real-world enterprise environment, configuration files rarely remain the same across all servers. Consider a web application deployed across three environments:

- Development
- Testing
- Production

Although each environment uses the same application, the configuration values are different.
For example:

Configuration	Development	Testing	Production
Application Name	Employee Portal	Employee Portal	Employee Portal
Environment	Development	Testing	Production
Server Name	dev-web01	test-web01	prod-web01
Database Server	dev-db	test-db	prod-db
Port	8080	8081	80

Maintaining separate configuration files for every environment quickly becomes difficult and error-prone. Ansible solves this problem using **Jinja2 Templates**.
A template is a text file that contains variables instead of hardcoded values. During playbook execution, Ansible replaces these variables with actual values and generates the final configuration file on the managed node.
Templates make playbooks reusable, easier to maintain, and suitable for multiple environments.

Prerequisites

Before beginning this practical, ensure that:

- Apache HTTP Server (httpd) is installed.
- The Apache service is running.
- Practical 8 (Handlers) has been completed.
- The managed node is reachable.
- The inventory file is configured correctly.
- The Ansible ping module returns **pong**.

Verify connectivity.

ansible all -i inventory.ini -m ping

What is a Template?
A template is a file that contains placeholders instead of fixed values.
For example, instead of writing:

<h1>Welcome to Employee Portal</h1>

you write:

<h1>Welcome to {{ application_name }}</h1>

When the playbook executes, Ansible replaces:

{{ application_name }}

with the value defined in the playbook.
For example,
If

application_name: Employee Portal

The generated file becomes

<h1>Welcome to Employee Portal</h1>

This process is known as **Template Rendering**.

Template vs Copy Module

Copy Module	Template Module
Copies the file exactly as it exists	Generates the file dynamically
Does not process variables	Replaces Jinja2 variables
Suitable for static files	Suitable for configuration files
File remains unchanged	File content changes depending on variables

Project Structure
Before starting, create the following project structure.

```
template-demo/
|
├── inventory.ini
├── website.yml
└── templates/
    └── index.html.j2
```

Step-by-Step Procedure

Step 1 – Create a Project Directory
Create a directory for this practical.

mkdir template-demo

cd template-demo

Explanation
As automation projects grow, multiple playbooks, templates, variables, and configuration files are created.Keeping all related files inside a dedicated project directory makes the project easier to organize, maintain, and share.

Step 2 – Create the Templates Directory
Create a directory named **templates**.

mkdir templates

Explanation
By convention, Ansible stores Jinja2 template files inside a directory named templates.
This convention is also followed by Ansible Roles, making projects easier to understand.

Step 3 – Create the Template File
Create the template.

vi templates/index.html.j2

Add the following content.

```
<!DOCTYPE html>

<html>


<head>

   <title>{{ application_name }}</title>

</head>


<body>
```

```
<h1>{{ application_name }}</h1>

<hr>

<p><strong>Environment :</strong> {{ environment }}</p>

<p><strong>Server Name :</strong> {{ ansible_hostname }}</p>

<p><strong>Operating System :</strong> {{ ansible_distribution }}</p>

<p><strong>Kernel Version :</strong> {{ ansible_kernel }}</p>

<p><strong>IP Address :</strong> {{ ansible_default_ipv4.address }}</p>

<p><strong>Total Memory :</strong> {{ ansible_memtotal_mb }} MB</p>

</body>

</html>
```

Explanation
This file is **not** copied directly to the managed node.
Instead, Ansible reads the file and searches for everything enclosed within:

{{ }}

Each placeholder represents a variable.
Some variables such as

application_name

environment

are defined by the playbook.
Other variables such as

ansible_hostname

ansible_distribution

ansible_kernel

ansible_default_ipv4.address

are **Ansible Facts**, automatically collected before task execution.
During playbook execution, Ansible replaces every placeholder with its corresponding value.

Step 4 – Create the Playbook
Create the playbook.

vi website.yml

Add the following content.

```
---

- name: Deploy Dynamic Web Page


  hosts: all


  become: yes


  vars:


    application_name: OCI DevOps Training


    environment: Development
```

```
 tasks:


   - name: Deploy web page template

     template:

       src: templates/index.html.j2

       dest: /var/www/html/index.html

     notify: Restart Apache


 handlers:


   - name: Restart Apache

     service:

       name: httpd

       state: restarted
```

Explanation
The playbook begins by defining two variables.

application_name: OCI DevOps Training

environment: Development

These variables will replace the corresponding placeholders inside the template.
The **template** module then reads the Jinja2 template, renders it, and writes the generated HTML page to

/var/www/html/index.html

If the generated file differs from the existing file, the task reports **changed**.
The handler is then notified to restart Apache.

Step 5 – Validate the Playbook
Before executing any playbook, validate its syntax.

ansible-playbook -i inventory.ini website.yml --syntax-check

Expected output

playbook: website.yml

Explanation
Syntax checking verifies the YAML structure without making any changes to the managed node.
It is considered a best practice to perform a syntax check before executing production playbooks.

Step 6 – Execute the Playbook
Run

ansible-playbook -i inventory.ini website.yml

Expected output

TASK [Deploy web page template]

changed: [web1]

RUNNING HANDLER [Restart Apache]

changed: [web1]

Explanation
During execution, Ansible performs the following operations:

1. Connects to the managed node.
2. Collects system facts.
3. Reads the template file.
4. Replaces all variables.
5. Generates the final HTML page.
6. Copies the page to the managed node.
7. Detects that the file has changed.
8. Executes the Apache restart handler.

Step 7 – Verify the Generated Web Page
Open a web browser and navigate to

http://<Managed_Node_IP>

Expected output

OCI DevOps Training

Environment : Development

Server Name : server01

Operating System : Oracle Linux

Kernel Version : ...

IP Address : ...

Total Memory : ...

Explanation
Notice that the HTML page now contains live system information.
The values were not manually entered—they were generated automatically using Ansible facts during playbook execution.

Step 8 – Modify a Variable
Open the playbook.

vi website.yml

Change

environment: Development

to

environment: Production

Execute the playbook again.

ansible-playbook -i inventory.ini website.yml

Explanation
Only one variable was modified.
However, Ansible automatically regenerates the entire HTML page with the updated value.
Since the generated file changed, the handler is triggered and Apache restarts.
No changes to the template file are required.

Step 9 – Verify Idempotency
Execute the playbook once more without making any changes.

ansible-playbook -i inventory.ini website.yml

Expected output

TASK [Deploy web page template]

ok: [web1]

PLAY RECAP

changed=0

Explanation
Since the generated file already matches the template output, no changes are required.
Because no changes occurred:

- the handler is not notified,
- Apache is not restarted,
- the playbook completes successfully.
- This demonstrates Ansible's idempotent behavior.

Practical 15– Automating Repetitive Tasks Using Ansible Loops

Objective

The objective of this practical is to learn how to use loops in Ansible to execute the same task multiple times with different input values. You will understand the loop keyword, iterate over lists, create multiple users, directories, and packages, and learn how loops improve the efficiency and readability of Ansible playbooks.

Overview

In many automation scenarios, administrators need to perform the same task repeatedly.
For example:

- Install multiple software packages.
- Create multiple users.
- Create several directories.
- Copy multiple files.
- Start multiple services.

One approach is to write the same task repeatedly.

```
- name: Install Git
  dnf:
    name: git
    state: present

- name: Install HTTPD
  dnf:
    name: httpd
    state: present

- name: Install wget
  dnf:
    name: wget
    state: present
```

Although this works, it results in repetitive code that becomes difficult to maintain.
Ansible provides **Loops**, allowing a single task to be executed multiple times using different values.
The same playbook can be written as:

```
- name: Install packages
  dnf:
    name: "{{ item }}"
    state: present
  loop:
    - git
    - httpd
    - wget
```

Instead of writing three separate tasks, a single task is executed three times.
Loops reduce code duplication, simplify maintenance, and make playbooks easier to understand.

Prerequisites
Before beginning this practical, ensure that:

- The managed node is reachable.
- The inventory file is configured correctly.
- Python is installed on the managed node.
- The Ansible ping module returns **pong**.

What is a Loop?
A loop instructs Ansible to execute the same task repeatedly using different values.
Instead of creating multiple identical tasks, one task is executed multiple times.
The value being processed during each iteration is stored in a special variable called:

item

Suppose the loop contains:

loop:

- git

- wget

- httpd

Execution happens like this:

Iteration	item
1	git
2	wget
3	httpd

During each iteration, {{ item }} is replaced with the current value.

Understanding the Loop Keyword
General syntax

```
tasks:

- name: Task Name
  module:
    parameter: "{{ item }}"

  loop:
    - value1
    - value2
    - value3
```

Step-by-Step Procedure
Step 1 – Create a New Playbook
Create a file named

vi loop-demo.yml

Explanation
This playbook demonstrates how Ansible executes one task repeatedly using the loop keyword. Rather than creating multiple identical tasks, we will execute one task several times.

Step 2 – Display Multiple Messages
Add the following playbook.

- name: Loop Demonstration

hosts: all

tasks:

- name: Display package names

debug:

```
    msg: "{{ item }}"


  loop:

    - git

    - wget

    - httpd

    - vim
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Explanation
The debug module executes once for every value inside the loop.
During execution:
First iteration

item = git

Second iteration

item = wget

Third iteration

item = httpd

Fourth iteration

item = vim

Notice that the task itself is written only once.
Ansible automatically repeats it for each value.

Step 3 – Install Multiple Packages
Modify the playbook.

```
---

- name: Install Packages
```

```
hosts: all

become: yes

tasks:

  - name: Install required packages

    dnf:

      name: "{{ item }}"

      state: present

    loop:

      - git

      - wget

      - vim
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Verify

```
ansible all -i inventory.ini -m command -a "rpm -q git wget vim"
```

Explanation
The dnf module installs one package during each loop iteration.
Execution flow

Iteration 1

Install git

↓

Iteration 2

Install wget

↓

Iteration 3

Install vim

Instead of writing three installation tasks, one task performs all installations.

Step 4 – Create Multiple Directories
Modify the playbook.

```
---

- name: Create Directories

  hosts: all

  tasks:

    - name: Create training directories
```

```
    file:

      path: "{{ item }}"

      state: directory

    loop:

      - /tmp/dev
      - /tmp/test
      - /tmp/prod
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Verify

```
ansible all -i inventory.ini -m command -a "ls -ld /tmp/dev /tmp/test /tmp/prod"
```

Explanation
Each directory path becomes the value of item.
Iteration example

Iteration 1

item=/tmp/dev

↓

Create directory

↓

Iteration 2

item=/tmp/test

↓

Create directory

↓

Iteration 3

item=/tmp/prod

One task creates three different directories.

Step 5 – Create Multiple Files
Modify the playbook.

```
---

- name: Create Files

  hosts: all

  tasks:
```

```
  - name: Create log files

    file:

      path: "{{ item }}"

      state: touch

    loop:

      - /tmp/dev/app.log

      - /tmp/test/app.log

      - /tmp/prod/app.log
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Verify

```
ansible all -i inventory.ini -m command -a "ls -l /tmp/dev /tmp/test /tmp/prod"
```

Explanation
The same file module executes three times using different file paths.

Step 6 – Create Multiple Users
Modify the playbook.

```
---

- name: Create Users

  hosts: all
```

```
  become: yes

  tasks:

    - name: Create application users

      user:

        name: "{{ item }}"

        state: present

      loop:

        - developer1

        - developer2

        - tester1
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Verify

```
ansible all -i inventory.ini -m command -a "id developer1"
```

Explanation
Each username is processed independently.
Execution order

developer1

↓

developer2

↓

tester1

One task creates three user accounts.

Step 7 – Use Variables with Loops
Instead of defining the list directly inside the loop, create variables.

```
---

- name: Loop with Variables

  hosts: all

  vars:

    packages:

      - git

      - wget

      - vim

      - tree

  become: yes
```

```
 tasks:

   - name: Install packages

     dnf:

       name: "{{ item }}"

       state: present

     loop: "{{ packages }}"
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Explanation
Instead of embedding the package list inside the task, the list is stored in a variable.
Benefits include:

- Easier maintenance
- Better readability
- Reusability across multiple tasks
- Centralized configuration

Step 8 – Loop Through Ansible Facts
Create the following playbook.

```
---

- name: Display Information

  hosts: all
```

```
  vars:

    information:

      - "{{ ansible_hostname }}"
      - "{{ ansible_distribution }}"
      - "{{ ansible_kernel }}"
      - "{{ ansible_processor_vcpus }}"

  tasks:

    - name: Display system information

      debug:

        msg: "{{ item }}"

      loop: "{{ information }}"
```

Execute

```
ansible-playbook -i inventory.ini loop-demo.yml
```

Explanation
The loop iterates through values obtained from Ansible Facts rather than hardcoded strings.
This demonstrates that loops can process dynamic data collected from the managed node.

Step 9 – Observe Idempotency
Execute any of the previous playbooks again without making changes.

ansible-playbook -i inventory.ini loop-demo.yml

Expected output

ok: [web1]

changed=0

Explanation
Ansible checks each iteration independently.
If a package is already installed or a directory already exists, Ansible reports ok instead of changed.
This demonstrates that loops preserve Ansible's idempotent behavior.

Practical 16 – Using Conditionals in Ansible (when, failed_when, and changed_when)

Objective

The objective of this practical is to learn how to control task execution in Ansible using conditional statements. You will execute tasks based on system facts, verify resource states before performing operations, register command outputs for decision-making, and customize how Ansible reports task failures and task changes.

Learning Outcomes

After completing this practical, you will be able to:

- Use the when statement to execute tasks conditionally.
- Execute tasks based on Ansible facts.
- Use the stat module with conditional execution.
- Register task output using the register keyword.
- Use registered variables in conditional statements.
- Customize task failures using failed_when.
- Control task status using changed_when.

Execute a Task Conditionally Using when

Overview

The when statement allows a task to execute only when a specified condition evaluates to **True**. This is one of the most commonly used features in Ansible because it enables playbooks to make decisions based on system facts or previously collected information.

In this activity, you will display a message only if the managed node is running Oracle Linux.

Step 1 – Create a Playbook

Create a new playbook.

vi when-demo.yml

Add the following content.

```
---

- name: Execute Task Conditionally

  hosts: all

  gather_facts: yes
```

```
 tasks:


   - name: Display operating system

     debug:

       msg: "This server is running Oracle Linux."


     when: ansible_distribution == "OracleLinux"
```

Save the file.

Explanation

Before executing any task, Ansible gathers system information (facts) from the managed node.

One of these facts is:

ansible_distribution

which contains the Linux distribution name.

The when statement evaluates the condition:

ansible_distribution == "OracleLinux"

If the condition is true, the task executes.

If the condition is false, the task is skipped.

Step 2 – Execute the Playbook

Run the playbook.

ansible-playbook -i inventory.ini when-demo.yml

Expected Output

TASK [Display operating system] **********

```
ok: [oraclelinux] => {
    "msg": "This server is running Oracle Linux."
}
```

Verification

The task should execute successfully because the managed node is Oracle Linux.

If the playbook were executed against another Linux distribution, this task would be skipped automatically.

Create a Directory Only If It Does Not Exist

Overview

Before creating files or directories, it is often useful to verify whether they already exist.

The stat module retrieves information about files and directories without modifying the system.

In this activity, you will create a directory only when it does not already exist.

Step 1 – Create a New Playbook

vi directory-check.yml

Add the following content.

```
---
- name: Create Directory Conditionally
  hosts: all

  tasks:

    - name: Check if directory exists
      stat:
        path: /opt/application
```

```
    register: directory_status


  - name: Create directory
    file:
      path: /opt/application
      state: directory


    when: not directory_status.stat.exists
```

Save the playbook.

Explanation

The first task uses the stat module to determine whether the directory already exists.

The result is stored in the variable:

directory_status

The second task evaluates:

when: not directory_status.stat.exists

If the directory is absent, the task creates it.

If the directory already exists, the task is skipped.

This prevents unnecessary operations and makes the playbook idempotent.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini directory-check.yml

Expected Output (First Execution)

```
TASK [Check if directory exists] **********
ok

TASK [Create directory] ******************
changed
```

Execute the Playbook Again

ansible-playbook -i inventory.ini directory-check.yml

```
TASK [Check if directory exists] **********
ok

TASK [Create directory] ******************
skipping
```

Verification

Verify the directory.

ansible all -i inventory.ini -m command -a "ls -ld /opt/application"

The directory should exist.

Notice that running the playbook multiple times does not recreate the directory because of the conditional statement.

Install and Start Apache Conditionally

Overview

Many automation tasks require decisions based on the current state of the system.

In this activity, you will:

- Check whether Apache (httpd) is already installed.
- Install it only if it is missing.
- Start the Apache service only after confirming that the package exists.

This demonstrates how to combine the package_facts, register, and when features to build intelligent playbooks.

Step 1 – Create the Playbook

vi apache-install.yml

Add the following content.

```
---
- name: Install Apache Conditionally
  hosts: all
  become: yes

  tasks:

    - name: Gather installed package information
      package_facts:

    - name: Install Apache
      dnf:
        name: httpd
        state: present
      when: "'httpd' not in ansible_facts.packages"

    - name: Check whether Apache is installed
      command: rpm -q httpd
      register: apache_status
      changed_when: false

    - name: Start Apache Service
      service:
```

```
    name: httpd
    state: started
    enabled: yes
  when: apache_status.rc == 0
```

Save the playbook.

Explanation

The first task collects information about all installed packages on the managed node.

The second task checks whether the package list contains httpd.

If Apache is not installed, Ansible installs it.

The third task executes:

rpm -q httpd

The output is stored in the variable:

apache_status

Since this command only retrieves information, it does not change the system.

Therefore,

changed_when: false

ensures that Ansible reports the task as **OK** instead of **Changed**.

Finally, the last task starts the Apache service only when the command succeeds.

The condition:

when: apache_status.rc == 0

checks the command's return code.

A return code of **0** indicates that Apache is installed.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini apache-install.yml

Expected Output

```
TASK [Gather installed package information] ***********
ok

TASK [Install Apache] ******************************
changed

TASK [Check whether Apache is installed] **************
ok

TASK [Start Apache Service] ************************
changed
```

If Apache is already installed, the installation task will be skipped automatically.

Step 3 – Verify Apache

Verify the installed package.

ansible all -i inventory.ini -m command -a "rpm -q httpd"

Verify that the service is running.

ansible all -i inventory.ini -m command -a "systemctl status httpd --no-pager"

Verification

Confirm that:

- Apache is installed.
- The Apache service is running.
- The service is enabled to start automatically after system reboot.

Execute Tasks Using Multiple Conditions

Overview

In many automation scenarios, a task should execute only when multiple conditions are satisfied. Ansible allows multiple conditions to be specified using the when statement. All conditions must evaluate to **True** before the task is executed.

In this activity, you will display a message only if the managed node is running Oracle Linux and has more than 1 GB of memory.

Step 1 – Create a New Playbook

Create a playbook named multiple-conditions.yml.

vi multiple-conditions.yml

Add the following content.

```
---

- name: Execute Tasks Using Multiple Conditions

  hosts: all

  gather_facts: yes



  tasks:



    - name: Display server information

      debug:

        msg: "The server meets all deployment requirements."



      when:

        - ansible_distribution == "OracleLinux"

        - ansible_memtotal_mb > 1024
```

Save the file.

Explanation
This playbook uses two conditions:

- ansible_distribution == "OracleLinux" verifies that the managed node is running Oracle Linux.
- ansible_memtotal_mb > 1024 verifies that the system has more than 1 GB of physical memory.

Since both conditions are listed under the same when statement, Ansible treats them as a logical **AND** operation. The task executes only if both conditions are true.

Step 2 – Execute the Playbook
Run the following command.

ansible-playbook -i inventory.ini multiple-conditions.yml

Expected Output

TASK [Display server information] **********

ok: [oraclelinux] => {

"msg": "The server meets all deployment requirements."

}

Verification
Verify that the message is displayed.
To view the available memory reported by Ansible, execute:

ansible all -i inventory.ini -m setup -a "filter=ansible_memtotal_mb"

Ensure that the reported memory satisfies the condition specified in the playbook.

Customize Task Failure and Change Reporting

Overview

By default, Ansible marks a task as **failed** when a command returns a non-zero exit code and marks command execution as **changed**. In some situations, this default behavior is not desirable.

In this activity, you will learn how to use:

- failed_when to customize task failure.
- changed_when to control whether a task is reported as changed.

Step 1 – Create a New Playbook

Create a playbook named task-status.yml.

vi task-status.yml

Add the following content.

```
---

- name: Customize Task Status

  hosts: all


  tasks:


    - name: Check Apache Service

      command: systemctl status httpd

      register: service_status

      failed_when: false

      changed_when: false


    - name: Display Service Status

      debug:

        var: service_status.rc
```

Save the playbook.

Explanation
The first task checks the status of the Apache service.
Normally:

- If the service is stopped, the command returns a non-zero exit code.
- Ansible treats the task as failed.

However,

failed_when: false

prevents the playbook from failing.
Similarly,

changed_when: false

ensures that Ansible reports the task as **OK** rather than **Changed**, since the command only retrieves information and does not modify the system.
The second task displays the command's return code stored in the registered variable.

Step 2 – Execute the Playbook
Run the following command.

ansible-playbook -i inventory.ini task-status.yml

Expected Output

TASK [Check Apache Service] **********

ok: [oraclelinux]

TASK [Display Service Status] **********

ok: [oraclelinux] => {

"service_status.rc": 0

}

Verification
Stop the Apache service.

ansible all -i inventory.ini -b -m service -a "name=httpd state=stopped"

Run the playbook again.
Notice that:

- The playbook continues executing.
- The task is still reported as **OK**.
- The return code changes, indicating that the service is no longer running.

Restart the service.

ansible all -i inventory.ini -b -m service -a "name=httpd state=started"

Combine register, when, failed_when, and changed_when

Overview

In this activity, you will combine the concepts learned throughout this practical to build a playbook that makes decisions based on the current state of the managed node.

The playbook will:

- Check whether Apache is installed.
- Register the command output.
- Start the Apache service only if it is installed.
- Report informational tasks as **OK**.
- Prevent unnecessary task failures.

Step 1 – Create a New Playbook

Create a playbook named conditional-demo.yml.

vi conditional-demo.yml

Add the following content.

```
---

- name: Demonstrate Conditional Execution

  hosts: all

  become: yes



  tasks:
```

```
  - name: Verify Apache Installation
    command: rpm -q httpd
    register: apache_status
    failed_when: false
    changed_when: false

  - name: Display Package Status
    debug:
      msg: "Apache is installed."
    when: apache_status.rc == 0

  - name: Start Apache Service
    service:
      name: httpd
      state: started
    when: apache_status.rc == 0

  - name: Display Warning
    debug:
      msg: "Apache is not installed."
    when: apache_status.rc != 0
```

Save the playbook.

Explanation

The first task checks whether the Apache package is installed using the rpm -q httpd command.

The output of the command is stored in the apache_status variable using the register keyword.

The task uses:

failed_when: false

to prevent the playbook from stopping if Apache is not installed.

It also uses:

changed_when: false

because checking the package status does not modify the system.

The second task displays a confirmation message only if the package exists.

The third task starts the Apache service only when the package is installed.

The final task displays a warning if Apache is not present on the managed node.

This approach allows the playbook to make intelligent decisions based on the current system state.

Step 2 – Execute the Playbook

Run the playbook.

ansible-playbook -i inventory.ini conditional-demo.yml

Expected Output

TASK [Verify Apache Installation] **********

ok

TASK [Display Package Status] **********

ok

TASK [Start Apache Service] **********

changed

TASK [Display Warning] ****************************

skipping

If Apache is not installed, the output changes to:

TASK [Verify Apache Installation] ******************

ok

TASK [Display Package Status] **********************

skipping

TASK [Start Apache Service] ************************

skipping

TASK [Display Warning] ****************************

ok

Verification

Verify that the Apache service is running.

ansible all -i inventory.ini -m command -a "systemctl status httpd --no-pager"

Confirm that:

- The Apache package is detected correctly.
- The service starts only when it is installed.
- The playbook completes successfully without unexpected failures.

Best Practices

- Use when to execute tasks only when necessary.
- Gather and use Ansible facts instead of hardcoding system information.
- Use the stat module before creating or modifying files and directories.
- Register command output when subsequent tasks depend on the results of previous tasks.
- Use failed_when only when you intentionally want to override Ansible's default failure behavior.
- Use changed_when: false for informational commands that do not modify the managed node.
- Test conditional logic thoroughly to ensure tasks are executed only when intended.

Practical 17 – Using Tags for Selective Task Execution

Objective

The objective of this practical is to learn how to use Ansible tags to control which tasks are executed within a playbook. Tags allow administrators to run specific tasks without executing the entire playbook, making automation more flexible, efficient, and easier to maintain.

Learning Outcomes

After completing this practical, you will be able to:

- Understand the purpose of Ansible tags.
- Assign tags to individual tasks.
- Execute specific tasks using tags.
- Skip tasks using tags.
- List available tags in a playbook.
- Apply multiple tags to a task.

Activity 1 – Assign Tags to Individual Tasks

Step 1 – Create a Playbook

Create a new playbook named tags-demo.yml.

vi tags-demo.yml

Add the following content.

```
---
- name: Demonstrate Ansible Tags
  hosts: all
  become: yes

  tasks:

    - name: Install Apache
      dnf:
        name: httpd
        state: present
      tags:
        - install

    - name: Start Apache Service
      service:
        name: httpd
        state: started
        enabled: yes
      tags:
        - service

    - name: Display Hostname
      command: hostname
      register: hostname_output
      changed_when: false
      tags:
        - info
```

```
name: Show Hostname
    debug:
      var: hostname_output.stdout
    tags:
      - info
```

Explanation

Each task is assigned a tag that identifies its purpose.

- install is used for package installation.
- service is used for service management.
- info is used for information gathering.

Tags allow these tasks to be executed independently without modifying the playbook.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini tags-demo.yml

Since no tag is specified, Ansible executes all tasks in the playbook.

Verification

Verify that:

- Apache is installed.
- Apache service is started.
- The hostname is displayed.

Activity 2 – Execute Tasks Using a Specific Tag

Step 1 – Run Only the Installation Task

Execute the playbook using the install tag.

ansible-playbook -i inventory.ini tags-demo.yml --tags install

Explanation

The --tags option instructs Ansible to execute only the tasks associated with the specified tag.
All other tasks are skipped.

Verification

Observe that only the **Install Apache** task is executed.

Step 2 – Execute Only the Service Task

ansible-playbook -i inventory.ini tags-demo.yml --tags service

Verification

Only the task responsible for managing the Apache service should execute.

Activity 3 – Execute Multiple Tags

Step 1 – Run More Than One Tag

Execute the following command.

ansible-playbook -i inventory.ini tags-demo.yml --tags "install,service"

Explanation

Multiple tags can be specified as a comma-separated list.
Ansible executes all tasks that match either tag.

Verification

Verify that:

- Apache installation task executes.
- Apache service task executes.
- Information gathering tasks are skipped.

Activity 4 – Skip Tagged Tasks

Step 1 – Skip Information Gathering Tasks

Execute the playbook.

ansible-playbook -i inventory.ini tags-demo.yml --skip-tags info

Explanation

The --skip-tags option excludes tasks associated with the specified tag while executing all remaining tasks.

This is useful when certain tasks are unnecessary for a particular execution.

Verification

Confirm that:

- Apache installation task executes.
- Apache service task executes.
- Hostname-related tasks are skipped.

Activity 5 – Assign Multiple Tags to a Task

Step 1 – Modify the Playbook

Edit the playbook.

vi tags-demo.yml

Modify the Apache service task.

```
- name: Start Apache Service
  service:
    name: httpd
    state: started
    enabled: yes
  tags:
    - service
    - apache
```

Explanation

A task can have multiple tags.

This allows the same task to be executed under different categories.

Step 2 – Execute the Apache Tag

ansible-playbook -i inventory.ini tags-demo.yml --tags apache

Verification

Only the Apache service task should execute.

Activity 6 – Display Available Tags

Step 1 – List All Tags

Execute the following command.

ansible-playbook -i inventory.ini tags-demo.yml --list-tags

Explanation

This command displays every tag defined within the playbook without executing any tasks.

It is useful for administrators who want to understand the available execution options before running a playbook.

Verification

Example output:

playbook: tags-demo.yml

play #1 (all): Demonstrate Ansible Tags TAGS: []
TASK TAGS: [apache, info, install, service]

.

Best Practices

- Assign meaningful tag names that describe the purpose of the task.
- Use consistent naming conventions throughout all playbooks.
- Group related tasks using common tags.
- Use --list-tags to review available tags before execution.
- Use --skip-tags to exclude unnecessary tasks during maintenance activities.

Practical 18 – Error Handling Using Blocks, Rescue, and Always

Objective

The objective of this practical is to learn how to handle errors in Ansible using block, rescue, and always. You will group related tasks, recover from failures gracefully, and execute cleanup or notification tasks regardless of whether the playbook succeeds or fails.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1 through Practical 12 have been completed successfully.
- The Ansible Control Node is configured.
- The Oracle Linux managed node is reachable.
- The inventory file is correctly configured.

Verify connectivity.

```
ansible all -i inventory.ini -m ping
```

Activity 1 – Group Tasks Using a Block

Step 1 – Create a Playbook

Create a playbook named block-demo.yml.

```
vi block-demo.yml
```

Add the following content.

```
---
- name: Demonstrate Block
  hosts: all
  become: yes

  tasks:

    - name: Install and Start Apache
      block:

        - name: Install Apache
          dnf:
            name: httpd
            state: present

        - name: Start Apache Service
          service:
            name: httpd
            state: started
            enabled: yes
```

Explanation

The block keyword groups multiple related tasks into a single logical unit. In this example, installing Apache and starting the Apache service are grouped because they belong to the same operation.

Step 2 – Execute the Playbook

```
ansible-playbook -i inventory.ini block-demo.yml
```

Verification

Verify that Apache is installed and the service is running.

```
ansible all -i inventory.ini -m command -a "systemctl status httpd --no-pager"
```

Activity 2 – Recover from a Failed Task Using rescue

Step 1 – Create a Playbook

Create a playbook named rescue-demo.yml.

vi rescue-demo.yml

Add the following content.

```
---
- name: Demonstrate Rescue
  hosts: all
  become: yes

  tasks:

    - block:

        - name: Install Invalid Package
          dnf:
            name: apache-invalid
            state: present

      rescue:

        - name: Display Failure Message
          debug:
            msg: "Package installation failed. Please verify the package name."
```

Explanation

The playbook intentionally attempts to install a package that does not exist. Since the installation fails, Ansible skips the remaining tasks in the block and immediately executes the tasks defined under the rescue section. This approach allows the playbook to recover from failures instead of terminating unexpectedly.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini rescue-demo.yml

Verification

Verify that the package installation fails and the custom failure message is displayed.

Activity 3 – Execute Cleanup Tasks Using always

Step 1 – Modify the Previous Playbook

Edit the playbook.

vi rescue-demo.yml

Modify it as shown below.

```
---
- name: Demonstrate Always
  hosts: all
  become: yes

  tasks:

    - block:

        - name: Install Invalid Package
          dnf:
            name: apache-invalid
            state: present

      rescue:
```

```
    - name: Display Failure Message
      debug:
        msg: "Package installation failed."

  always:

    - name: Display Completion Message
      debug:
        msg: "Playbook execution completed."
```

Explanation

The always section is executed regardless of whether the tasks in the block succeed or fail.
It is commonly used for:

- Cleanup operations
- Closing connections
- Removing temporary files
- Logging
- Sending notifications

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini rescue-demo.yml

Verification

Observe that:

- The package installation fails.
- The rescue task executes.
- The task in the always section also executes.

Activity 4 – Use Block with Multiple Tasks

Step 1 – Create a Playbook

Create a playbook named block-service.yml.
vi block-service.yml
Add the following content.

```
---
- name: Configure Apache
  hosts: all
  become: yes

  tasks:

    - block:

        - name: Install Apache
          dnf:
            name: httpd
            state: present

        - name: Create Web Directory
          file:
            path: /var/www/html/demo
            state: directory

        - name: Create Home Page
          copy:
            content: "<h1>Welcome to Apache</h1>"
            dest: /var/www/html/demo/index.html
```

```
    - name: Start Apache
      service:
        name: httpd
        state: started

  rescue:

    - debug:
        msg: "Apache configuration failed."

  always:

    - debug:
        msg: "Configuration task completed."
```

Explanation

This example groups multiple configuration tasks into a single block. If any task fails, the rescue section executes, and the always section runs after that.

Grouping related tasks improves readability and simplifies error handling.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini block-service.yml

Verification

Verify that:

- Apache is installed.
- The directory is created.
- The web page exists.
- Apache service is running.

ansible all -i inventory.ini -m command -a "ls -l /var/www/html/demo"

Activity 5 – Nested Blocks

Step 1 – Create a Playbook

Create a playbook named nested-block.yml.

vi nested-block.yml

Add the following content.

```
---
- name: Nested Block Example
  hosts: all
  become: yes

  tasks:

    - block:

        - debug:
            msg: "Starting application deployment."

        - block:

            - name: Install Apache
              dnf:
                name: httpd
                state: present

            - name: Start Apache
              service:
                name: httpd
```

```
        state: started

  rescue:

    - debug:
        msg: "Deployment failed."

  always:

    - debug:
        msg: "Deployment process finished."
```

Explanation

Blocks can be nested to organize complex automation tasks. This is useful when large playbooks contain multiple deployment phases, each requiring separate error handling.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini nested-block.yml

Verification

Confirm that the deployment messages are displayed and Apache is running.

Activity 6 – Understanding the Execution Flow

Step 1 – Create a Playbook

Create a playbook named execution-flow.yml.

vi execution-flow.yml

Add the following content.

```
---
- name: Execution Flow
  hosts: all

  tasks:

    - block:

        - debug:
            msg: "Task 1"

        - command: /bin/false

        - debug:
            msg: "Task 2"

      rescue:

        - debug:
            msg: "Executing Rescue Block"

      always:

        - debug:
            msg: "Executing Always Block"
```

Explanation

The command /bin/false always returns a non-zero exit status, causing the block to fail.

Notice the execution order:

1. Task 1 executes.
2. /bin/false fails.
3. Task 2 is skipped.
4. Rescue block executes.
5. Always block executes.

This demonstrates exactly how Ansible processes failures within a block.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini execution-flow.yml

Verification

Observe the sequence of task execution in the output and verify that it matches the expected execution flow.

Best Practices

- Group related tasks using block for better organization.
- Use rescue only for handling expected failures.
- Use always for cleanup, logging, or notification tasks.
- Keep blocks focused on a single logical operation.
- Test error-handling scenarios before deploying playbooks in production.

Practical 19 – Securing Sensitive Data Using Ansible Vault

Objective

The objective of this practical is to learn how to protect sensitive information in Ansible using Ansible Vault. You will create encrypted files, edit encrypted content, view encrypted data, use encrypted variables in playbooks, change vault passwords, and decrypt files when required.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1 through Practical 15 have been completed successfully.
- The Ansible Control Node is configured.
- The Oracle Linux managed node is reachable.
- Passwordless SSH authentication is configured.
- The inventory file is correctly configured.

Verify connectivity.

ansible all -i inventory.ini -m ping

Activity 1 – Create an Encrypted File

Step 1 – Create a Vault File

Execute the following command.

ansible-vault create secrets.yml

Enter a vault password when prompted.

Add the following content.

db_username: admin

db_password: Welcome@123

Save and exit the editor.

Explanation

The ansible-vault create command creates a new encrypted file. The contents are encrypted using the password supplied during creation, ensuring that sensitive information such as usernames, passwords, and API keys cannot be viewed in plain text.

Step 2 – Verify the File

Display the file contents.

cat secrets.yml

Verification

Verify that the file contents are encrypted and unreadable.

Activity 2 – View an Encrypted File

Step 1 – Display the File Contents

Execute the following command.

ansible-vault view secrets.yml

Enter the vault password.

Explanation

The view command temporarily decrypts the file and displays its contents without permanently decrypting it on disk.

Verification

Verify that the original variables are displayed correctly.

Activity 3 – Edit an Encrypted File

Step 1 – Edit the Vault File

Execute the following command.

ansible-vault edit secrets.yml

Update the password.

db_username: admin

db_password: Oracle@123

Save and exit.

Explanation

The edit command decrypts the file in memory, allows modifications, and encrypts the file again when it is saved. This prevents sensitive information from being stored as plain text.

Step 2 – Verify the Updated Content

ansible-vault view secrets.yml

Verification

Verify that the updated password is displayed.

Activity 4 – Use Vault Variables in a Playbook

Step 1 – Create a Playbook

Create a playbook named vault-demo.yml.

vi vault-demo.yml

Add the following content.

```
---
- name: Demonstrate Ansible Vault
  hosts: all

  vars_files:
    - secrets.yml

  tasks:

    - name: Display Database Username
      debug:
        var: db_username

    - name: Display Database Password
      debug:
        var: db_password
```

Explanation

The vars_files directive imports variables stored in the encrypted file. During playbook execution, Ansible decrypts the file after prompting for the vault password.

Step 2 – Execute the Playbook

ansible-playbook -i inventory.ini vault-demo.yml --ask-vault-pass

Enter the vault password when prompted.

Verification

Verify that the username and password stored in the encrypted file are displayed.

Activity 5 – Change the Vault Password

Step 1 – Rekey the Vault File

Execute the following command.

ansible-vault rekey secrets.yml

Enter:

- Current vault password
- New vault password
- Confirm the new vault password

Explanation

The rekey command changes the encryption password without modifying the file contents. This is useful when passwords need to be rotated as part of security policies.

Step 2 – Verify the New Password

View the encrypted file.

ansible-vault view secrets.yml

Enter the new vault password.

Verification

Verify that the file opens successfully using the new password.

Activity 6 – Decrypt the Vault File

Step 1 – Decrypt the File

Execute the following command.

ansible-vault decrypt secrets.yml

Enter the vault password.

Explanation

The decrypt command permanently removes encryption from the file. This should only be done when encryption is no longer required or when migrating sensitive information to another secure location.

Step 2 – Verify the Decryption

Display the file.

cat secrets.yml

Verification

Verify that the contents are displayed in plain text.

Best Practices

- Store only sensitive information in vault files.
- Use strong vault passwords and rotate them periodically.
- Never share vault passwords through unsecured channels.
- Keep encrypted files under version control instead of storing plain-text secrets.
- Use separate vault files for different environments such as development, testing, and production.
- Limit access to vault passwords to authorized administrators.